

Academic Thesis for the Attainment of the Degree of
Master of Science
at the Technical University of Munich

An Algorithmic Framework for Very Large-Scale Capacitated Vehicle Routing Problems

Examiner: Prof. Dr. Stefan Minner
Chair of Logistics and Supply Chain Management
Technische Universität München

Supervisor: Till Krieger, M.Sc.

Degree Program: Management & Technology Master

Submitted by:
Lucca Altendeitering
Stolzestr. 21, 44789 Bochum
Tel.: +49 162 7630077
Matriculation Number: 3782307

Date of Submission: 30.09.2026

An Abstract should not include more than 150 words.

Table of Contents

List of Figures	iv
List of Tables	v
Abbreviations	vi
1 Introduction	1
2 Literature Review	5
2.1 Problem Statement and Benchmark Instances	5
2.2 State-of-the-Art Solvers for Large-Scale CVRP	6
2.3 Scaling and Configuring Routing Heuristics	8
2.3.1 Parallel Search	8
2.3.2 Route Pools and Set Partitioning	8
2.3.3 Decomposition	9
2.3.4 Repairing Decomposition Boundaries	11
2.3.5 Adaptive Operator Selection and Algorithm Configuration . .	12
2.3.6 Learned Decomposition	13
2.4 Algorithmic Frameworks	13
2.5 Research Gap	15
3 Framework	17
3.1 Hierarchical Adaptive Operator Selection	18
3.2 Decomposition	22
3.3 Routing	24
3.4 Boundary-Guided Adaptive Iterated Local Search	25
3.5 Route Pool and Set Partitioning	31
3.6 Improvement	34
3.7 Parallelized Pipeline Orchestration	35
4 Computational Study	38
4.1 Experimental Design	38
4.1.1 Dataset	38
4.1.2 Framework Implementation	39
4.1.3 Hyperparameters	40
4.1.4 Replication Design	41

4.2	Results	43
4.2.1	Performance Comparison against State-of-the-Art Mono- lithic Solvers	43
4.2.2	Performance by Instance Characteristics	47
4.2.3	HAOS Weights	49
4.2.4	Convergence Behavior	49
5	Conclusion	50
5.1	Highlights	50
5.2	Limitations	50
5.3	Future Research	52
	Bibliography	53
	Appendix	62
1	Set Covering Formulation	62
2	XL Instance Detailed Results	63

List of Figures

3.1 Sequential Flowchart of DRISPI.	18
3.2 Illustrative BG-AILS Call on Instance XL-n1281-k29. Colorbar Applies to Panel (d) Only.	28
3.3 Per-Instance Mean Cost Improvement (a) and Paired BG-AILS Differences (b).	30
3.4 Two Consecutive DRISPI Iterations in One 8-Core Container (6 for DRI, 1 for BG-AILS, and 1 for SP). Block Lengths Are Illustrative.	35
4.1 Empirical CDF of the Mean-of- N Gap to the Current BKS.	45
4.2 Mean-of-3 Gap to the Current Best Known Solution (BKS) by Instance Characteristic, Across All 100 XL instances (Error Bars Are One Sample Standard Deviation).	49

List of Tables

2.1	Research Gap Matrix for Large-Scale Routing Methods.	16
3.1	Default Immediate and Deferred HAOS Rewards.	21
3.2	Inputs Used By Each Decomposition Method.	23
4.1	Hyperparameters of the Benchmark Configuration.	40
4.2	Forecast Standard Error and 95% Half-Width of $\bar{G}$ by Seed Count S . . .	42
4.3	DRISPI Mean Gap to Current BKS Comparison Against Benchmark Solvers, Sorted by Ascending Mean-of- N Gap Over All Instances. . . .	43
4.4	DRISPI's Largest Mean-of-3 Gap to Current BKS (Abbr. as per Sec- tion 4.1.1).	46
4.5	DRISPI's Best-of-3 Cost Matching or Beating a Published Reference. . .	47
A.1	Instance-Level Best-of-3 Results for DRISPI on the 100 XL Instances. .	63

Abbreviations

ALNS	Adaptive Large Neighborhood Search.
BKS	Best Known Solution.
CVRP	Capacitated Vehicle Routing Problem.
DRI	Decompose-Route-Improve.
HVRP	Heterogeneous Vehicle Routing Problem.
ILS	Iterated Local Search.
TSP	Traveling Salesman Problem.
VRPTW	Vehicle Routing Problem with Time Windows.

1 Introduction

Parcel carriers plan their delivery tours per depot. A single depot can serve several thousand customers a day. The instances that Arnold et al. (2019) derived from Belgian parcel operations range from a few thousand to 30,000 customers. Each such plan is an instance of the Capacitated Vehicle Routing Problem (CVRP). A homogeneous fleet of vehicles with limited capacity leaves a depot, visits every customer exactly once, and returns at minimum total travel cost (Toth and Vigo, 2014). Once customers and fleet are fixed, the route plan decides the travel cost. A small relative improvement then repeats on every tour of every day. The plan also has to be ready within hours. At this scale, a method is only useful if it reaches a good solution inside a fixed wall-clock budget.

Benchmark research has recently moved to the same scale. Queiroga et al. (2026) released the XL set of CVRPLib. The set contains 100 instances with 1,000 to 10,000 customers, generated under the scheme of the X instances (Uchoa et al., 2017). The release also reports initial Best Known Solution (BKS) values from two-hour runs of eight published solvers. With the release they opened the CVRPLib BKS Challenge on January 12, 2026. Any team can submit an improved solution for automatic verification. By early September 2026, the challenge had improved 99 of the 100 initial values. The XL set therefore combines a deliberately varied instance set under a common time budget with a reference that moves as the community improves it. This thesis also continues an unpublished project study that adapted the DRSCI framework of Kerscher et al. (2026) to the CVRP (Altendeitering and Bachmann, 2026). That study reached a mean gap of 0.53% to the initial XL BKS. That study drew its decomposition and routing configuration uniformly at random in every iteration. It left open whether a framework that learns this choice during the run could close the remaining distance to the monolithic solvers.

The CVRP is NP-hard (Lenstra and Rinnooy Kan, 1981). Branch-cut-and-price algorithms prove optimality for instances with a few hundred customers (Pecin et al., 2017; Pessoa et al., 2020). That size is an order of magnitude or more below the XL range. Solutions in that range come from metaheuristics. These methods trade the optimality guarantee for a feasible solution within a practical budget. On the XL set the strongest of these methods are monolithic. A monolithic solver keeps one complete solution, or a population of complete solutions, under a single search process. Of the 100 initial BKS values, AILS-II (Máximo et al., 2024) found 93, FILO2 (Accorsi and Vigo, 2024) six, and FILO (Accorsi and Vigo, 2021) one.

These solvers scale through engineering inside one solution. Granular neighbor-

hoods and selective vertex caching restrict move evaluation to promising arcs and recently changed vertices (Toth and Vigo, 2003; Accorsi and Vigo, 2021). The cost of neighborhood evaluation, the memory for move descriptors, and the number of iterations needed to reach a given quality still grow with the customer count (Arnold et al., 2019; Accorsi and Vigo, 2021). The benchmark results point to a second limit. Among the eight codes that Queiroga et al. (2026) compare, the hybrid genetic search HGS-CVRP (Vidal, 2022) has the best mean quality on instances with up to about 330 customers. AILS-II leads on the upper half of the X set and on XL. No single solver leads across the size range. Each also runs with one configuration on every instance. The XL generator varies depot position, customer distribution, demand, and route length on purpose. A configuration tailored to each instance would require a tuning campaign per instance.

Decomposition offers a way around both limits. Splitting the customer set into subproblems of a few hundred customers moves each subproblem into the size range where the strongest solvers are known. Independent subproblems can then be solved in parallel. The decompose-route-improve framework of Kerscher and Minner (2025) and its set-covering successor DRSCI (Kerscher et al., 2026) wrap existing routing software in this way. Santini et al. (2023) show that route-based decomposition improves both adaptive large neighborhood search and hybrid genetic search on the X instances. The benefit depends on the strategy and the instance. A poor decomposition can fail to help. Existing frameworks fix that choice before or outside the search. The choice is set by a calibrated rule (Queiroga et al., 2021; Santini et al., 2023), by offline tuning (Kerscher and Minner, 2025), by an offline-trained policy (Li et al., 2021; Ye et al., 2024), or by uniform randomization (Kerscher et al., 2026). None of them uses the solution costs observed on the instance at hand to decide which decomposition and which subsolver to apply next. This thesis develops DRISPI, a decomposition framework for very large CVRP instances. Its controller, hierarchical adaptive operator selection (HAOS), learns that choice within a single run. The primary research question is

RQ1: Can an adaptive decomposition framework that learns its decomposition and routing strategy within a single run match or outperform state-of-the-art monolithic solvers on very large CVRP instances within the same two-hour wall-clock limit per run?

We answer it on all 100 XL instances. Every instance is solved three times under one shared configuration with a two-hour wall-clock limit per run. We compare DRISPI’s mean gap to the current BKS against the published mean-of-60 results of the eight solvers of Queiroga et al. (2026). The five closest comparisons are tested with paired Wilcoxon signed-rank tests under a Holm correction. The comparison fixes the wall-clock limit but not the hardware. DRISPI runs on eight cores of an older processor. The published solvers each run on a single thread of a newer one. Section 5.2 quantifies that asymmetry.

The secondary research question concerns what the controller learns.

RQ2: Do the weight distributions HAOS learns differ systematically with instance characteristics, and do these differences give interpretable insight into instance structure?

We answer it from the logged weight trajectories of the same 300 runs, grouped by the instance attributes of the XL generator. The campaign contains no control arm with a frozen configuration. The answer to RQ2 is therefore descriptive. It shows what HAOS learns, not how much the learning itself adds.

This thesis makes four contributions. First, it develops DRISPI, an algorithm-agnostic framework that repeats a decompose-route-improve loop. The framework stores every route it generates in a persistent pool. It recombines that pool at scheduled intervals by set partitioning or set covering (SC/SP). Any CVRP solver that takes an instance and returns routes is a valid subsolver. The deployed configuration draws from AILS-II, FILO, FILO2, and PyVRP (Wouda et al., 2024). Second, it introduces HAOS, a hierarchical roulette-wheel controller in the tradition of adaptive large neighborhood search (Ropke and Pisinger, 2006). In every iteration, HAOS selects the number of subclusters, the demand weight of the dissimilarity metric, the clustering paradigm, the clustering method, and the subsolver. It credits each configuration twice. The first credit is immediate, for the solutions its own iteration produces. The second is deferred, for every route that SC/SP selects from the pool, traced back through a configuration tag stored on each route. Third, it proposes boundary-guided adaptive iterated local search (BG-AILS). This improvement stage perturbs the concatenated subproblem solutions across subcluster boundaries with a chain of cross-reconnect moves. It then hands the result to AILS-II, so that a short time budget goes to the seams the decomposition created. Fourth, it runs these stages as a parallel pipeline on eight cores. BG-AILS and the SC/SP step run asynchronously next to the decompose-route loop. A fitted wall-time model sizes the BG-AILS budget to the stage it overlaps with.

[TODO: Teaser BG-AILS: isolated one-iteration comparison against stock AILS-II on all 100 XL instances (Section 3.4).]

[TODO: Teaser main result: mean-of-3 gap to current BKS, rank among the eight published solvers, and the significance verdicts against AILS-II, FILO2, and FILO (Section 4.2.1).]

[TODO: Teaser individual results: XL-n4535-k1134 below its initial BKS, XL-n1094-k157 matching the current BKS (Table 4.5).]

[TODO: Teaser RQ2: one-sentence finding on HAOS weight convergence (Section 4.2.3).]

[TODO: Decide whether the computational study (all 100 XL instances against the current BKS, with the resource asymmetry disclosed) counts as a fifth contribution.]

The remainder of this thesis is structured as follows. Chapter 2 states the CVRP. It reviews the monolithic solvers that lead on the XL set and the strategies used to scale routing search. It then locates the research gap. Chapter 3 presents DRISPI stage by stage and closes with the parallel pipeline and its main loop. Chapter 4 describes the experimental design. It reports the comparison with the published solvers, the results by instance characteristic, the learned HAOS weights, and the convergence behavior. Chapter 5 answers both research questions, discusses the limitations of the study, and outlines directions for future research. Appendix 1 gives the set covering formulation. Appendix 2 lists the per-instance results.

2 Literature Review

The truck dispatching model of Dantzig and Ramser (1959) is the origin of the vehicle routing family. Subsequent research has treated the CVRP as its canonical member, with a homogeneous fleet, capacity constraints, and one visit per customer (Toth and Vigo, 2014). Vidal et al. (2020) and Archetti et al. (2026) review how that core model has been extended since.

This chapter states the CVRP and the benchmark instances used in the computational study (Section 2.1). It then moves from the solvers that lead on the XL instances (Section 2.2) to the techniques that scale and configure routing heuristics (Section 2.3). Section 2.4 covers the frameworks that wrap existing solvers, and Section 2.5 locates the remaining research gap.

2.1 Problem Statement and Benchmark Instances

The CVRP is defined on a complete undirected graph $G = (V, E)$, where $V = \{0\} \cup C$ is the set of locations. Node 0 represents the depot, and $C = \{1, \dots, n\}$ is the set of customers, so that $n = |C|$ is the customer count excluding the depot. Each edge $\{i, j\} \in E$ carries a non-negative travel cost $c_{ij} \geq 0$, and each customer $i \in C$ has a demand $q_i > 0$. A homogeneous fleet of vehicles with capacity Q is dispatched from and returns to the depot. A solution is a set of routes that start and end at the depot, visit every customer exactly once, and respect the capacity Q on every route. The objective is to minimize the total travel cost.

As a generalization of the Traveling Salesman Problem (TSP), the CVRP is NP-hard (Lenstra and Rinnooy Kan, 1981), and the number of feasible solutions grows factorially with n . Branch-cut-and-price algorithms prove optimality for instances with a few hundred customers (Pecin et al., 2017; Costa et al., 2019; Pessoa et al., 2020). That leaves them an order of magnitude short of the scale considered here. Heuristic and metaheuristic methods therefore trade the optimality guarantee for feasible solutions within a practical wall-clock budget.

Heuristic research on the CVRP has converged on a small number of benchmark sets. The X set of Uchoa et al. (2017) contains 100 instances with 100 to 1,000 customers, generated so that depot position, customer distribution, demand distribution, and average route length vary across the set. Queiroga et al. (2026) report that it became the primary benchmark for both exact and heuristic methods. It also

formed the basis of the CVRP track of the 12th DIMACS Implementation Challenge. Queiroga et al. (2022) add the XML set of 10,000 instances with 100 customers each, providing optimal solutions for almost all of them. Machine-learning heuristics can then be evaluated against absolute error rather than a moving reference.

Two collections reach far beyond 1,000 customers. The AGS set of Arnold et al. (2019) contains ten instances with 3,000 to 30,000 customers, derived from parcel delivery in Belgium. The AV set of Accorsi and Vigo (2024) contains 20 instances with 20,000 to 1,000,000 customers, generated from sampled addresses in Italian regions. Queiroga et al. (2026) observe that all 30 of these instances draw their demands uniformly from one to three units, and that most of them have long routes. None covers the range from 1,001 to 2,999 customers.

The XL set of Queiroga et al. (2026) closes that range with 100 instances of 1,000 to 10,000 customers. They are generated under a scheme very similar to that of the X set, varying the same four attributes. Queiroga et al. (2026) released the set with initial BKS values from a common solver benchmark, and opened the CVR-PLib BKS Challenge, a community competition that started on January 12, 2026. Chapter 4 uses the XL set and lists its attribute levels.

2.2 State-of-the-Art Solvers for Large-Scale CVRP

On the XL set, the methods that produced the initial BKS values are metaheuristics that search over a complete solution. We call a method monolithic when it maintains one complete feasible solution, or a population of them, under a single search process that can in principle modify any route. Neighborhoods may be localized or sparsified without affecting this classification. What falls outside it are independent customer partitions, solved separately and merged by an outer recombination layer.

Queiroga et al. (2026) ran AILS-II, FILO, FILO2, KGLS-XXL, HGS-CVRP, SISRs, LKH-3, and OR-Tools on every XL instance, with 60 seeds of two hours per code on a single thread. Of the 100 initial BKS values, AILS-II found 93, FILO2 six, and FILO one. The authors report HGS-CVRP, SISRs, LKH-3, and OR-Tools as an indication only, because these codes were not calibrated for instances with up to 10,000 customers. The two-hour limit is the budget previously used on X instances of at most 1,000 customers. The public HGS-CVRP code carries a disclaimer that it is designed for that range, and LKH-3 found no feasible solution on ten XL instances within the limit.

Iterated Local Search (ILS) alternates a perturbation of the current solution with a local search that returns it to a local optimum. An acceptance criterion then decides which local optimum the next perturbation starts from (Lourenço et al., 2003). Lourenço et al. (2003) argue that the perturbation strength governs the behavior of

the method. A perturbation that is too weak lets the local search fall back into the same local optimum, while one that is too strong turns the method into a random restart. The classical example calibrated to that tension is the double-bridge move of Martin et al. (1991), which a local search built from small moves cannot undo in one step.

Máximo et al. (2024) develop AILS-II as an adaptive ILS for large-scale CVRP instances. It extends AILS-PR (Máximo and Nascimento, 2021), which hybridizes adaptive ILS with path relinking and performs well up to 1,000 customers. AILS-II adapts the perturbation degree during the search, and in its second phase it draws the solution to be perturbed from an elite set. Its original experiments reach 30,000 customers on the AGS set, where it reported lower average gaps than both AILS-PR and HGS on all ten instances.

Accorsi and Vigo (2021) build FILO as an ILS with a simulated-annealing acceptance criterion. Granular neighborhoods (Toth and Vigo, 2003) restrict move evaluation to a subset of promising arcs, and selective vertex caching restricts it further to recently changed vertices. FILO2 (Accorsi and Vigo, 2024) carries this design to instances with up to one million customers. Both devices localize the search inside one full solution, rather than partitioning the customer set into independently solved subproblems.

Hybrid genetic search evolves a population of complete solutions and improves offspring by local search (Vidal et al., 2012). Vidal (2022) published HGS-CVRP as an open implementation for the CVRP with the SWAP* neighborhood, and Wouda et al. (2024) provide PyVRP, a separate implementation of the same paradigm. The comparison of Queiroga et al. (2026) uses the HGS-CVRP code.

Those same experiments show that the leading method changes with scale. On the XML instances and on the smaller half of the X set, up to about 330 customers, HGS-CVRP has the best average solution quality of the eight codes. AILS-II leads on the larger half of the X set and on XL. Queiroga et al. (2026) read this as a transition from population-based search, which evolves several complete solutions at once, to single-trajectory search.

The remaining codes follow other designs. SISRs is a ruin-and-recreate heuristic built on string removals (Christiaens and Vanden Berghe, 2020), and the knowledge-guided local search of Arnold and Sörensen (2019) was extended to very large instances as KGLS-XXL by Arnold et al. (2019). LKH-3 transforms constrained routing problems into a TSP with penalties, then applies the Lin-Kernighan-Helsgaun local search to the result (Helsgaun, 2017). An earlier variable neighborhood search had already addressed instances with up to 20,000 customers (Kytöjoki et al., 2007).

POPMUSIC (Queiroga et al., 2021) is the exception among the competitive large-scale methods. It reoptimizes subproblems extracted from a complete solution, and is therefore not monolithic in the sense above. Section 2.3.3 discusses it with the other decomposition methods.

2.3 Scaling and Configuring Routing Heuristics

Exact methods fail at scale because the solution space grows factorially, but local-search heuristics face a different cost. What grows with n is the effort of neighborhood evaluation, the memory for move descriptors and caches, and the number of iterations needed to reach a given quality (Arnold et al., 2019; Accorsi and Vigo, 2021).

The literature offers several responses. Parallel search spreads the effort over processors (Section 2.3.1), and route pools recombine routes found earlier by set covering or set partitioning (Section 2.3.2). Decomposition splits the customer set into smaller subproblems (Section 2.3.3), which leaves seams that later steps must repair (Section 2.3.4). Adaptive control and learned policies then decide which operator, configuration, or partition to apply (Sections 2.3.5 and 2.3.6).

2.3.1 Parallel Search

Crainic (2008) and Crainic (2019) classify parallel metaheuristics by how they split the search. Low-level parallelism accelerates neighborhood evaluation inside one trajectory. Multi-search strategies run several trajectories, either independently or as cooperating searches that exchange complete solutions. Crainic (2019) concentrates on the cooperative strategies, and describes those built on asynchronous exchanges as the most effective of them.

Applied to vehicle routing, these designs multiply processor effort on the full instance. Groër et al. (2011) combine heuristic local search with integer programming across as many as 129 processors. Jin et al. (2014) run cooperative tabu-search threads that share a common solution pool, where entering solutions are clustered by similarity so that the clusters can steer the threads between intensification and diversification.

In both methods, every worker still searches the full customer set. Parallelism changes how many neighborhoods or incumbents are examined per wall-clock second, without reducing the size of the problem each worker faces. Taillard (1993) instead partitions the customers geographically and solves the resulting subproblems with parallel iterative search. Solving customer subsets concurrently requires exactly that kind of decomposition layer, which Section 2.3.3 reviews.

2.3.2 Route Pools and Set Partitioning

Set covering and set partitioning over routes are older than local-search metaheuristics. Balinski and Quandt (1964) formulate delivery as an integer program over routes, and Garfinkel and Nemhauser (1969) treat set partitioning as set covering with equality constraints. Foster and Ryan (1976) solve the set partitioning model

of vehicle scheduling with the revised simplex method, restricting the columns to routes through radially contiguous locations. Their test problems range from 21 to 100 locations. Once a collection of feasible routes exists, selecting a subset that covers each customer once is an integer program whose size tracks the number of routes rather than n .

Rochat and Taillard (1995) store elite routes in an adaptive memory, draw new starting solutions from it probabilistically, and recombine the stored routes by set covering. The local search generating the routes and the model recombining them are already separate components there. Taillard et al. (2001) describe this scheme as adaptive memory programming and argue that several metaheuristics share it. A memory of solution components builds new starting solutions, a local search improves them, and the results update the memory. Kelly and Xu (1999) combine route generation by heuristics with a set partitioning model that selects among the generated routes.

Jourdan et al. (2009) classify such combinations of exact methods and metaheuristics in a taxonomy of hybrids. Several CVRP heuristics apply the pattern repeatedly during the search rather than once at the end. Subramanian et al. (2013) hybridize ILS with a set partitioning model over the routes it generates. Cavaliere et al. (2022) alternate LKH-3 local search with a restricted set partitioning program. Allowed to run 24 hours past its last improvement, it reports new best known solutions for large-scale CVRP instances. FSP4D, a finalist of the 12th DIMACS Implementation Challenge, follows the same alternation with a FILO-based fast ILS feeding a route pool (Accorsi et al., 2022a).

Arenas-Vasco et al. (2025) quantify what the pattern is worth. Their meta-analysis covers 30 set partitioning or set covering matheuristics for vehicle routing problems, selected from 54 eligible studies. The recombination step improved solution quality by 0.51% on average, with a 95% confidence interval of 0.41 to 0.61%. The authors describe the gain as modest but consistent across implementations.

Recombination can only choose among routes that some other component has already generated, so a pool dominated by the routes of one incumbent reproduces that incumbent. The diversity of the result is bounded by the diversity of the route generators feeding the pool. The recombination step itself decides nothing about how the instance is split, which solver generates the routes, or how either choice changes during a run.

2.3.3 Decomposition

A decomposition method splits the instance into smaller subproblems, solves each one, and reassembles a complete solution. How good that solution is depends on where the cuts fall, how often they are redrawn, and how the pieces are joined back together. Two questions organize the literature. The first is whether the customers

are grouped before or after a tour is built. The second is whether the groups are formed from customer coordinates or from the routes of an existing solution.

Cluster-first route-second answers the first question by partitioning the customers and then routing each cluster. The sweep algorithm of Gillett and Miller (1974) forms clusters by polar angle around the depot and solves a TSP on each. Fisher and Jaikumar (1981) assign customers to vehicle seeds through a generalized assignment problem before routing each assignment. Kerscher and Minner (2025) and Karaahmetoglu and Kim (2026) follow the same order with modern components, building a customer partition and routing its clusters with existing solvers.

Route-first cluster-second reverses that order. Beasley (1983) constructs a single tour through all customers, then partitions it into capacity-feasible routes, optimally for the given order. Prins (2004) embeds this split procedure in a genetic algorithm that encodes solutions as giant tours. The clustering step then operates on a customer sequence rather than on an unstructured point set.

The second question only arises once a partition budget is fixed, and it separates vertex-based from route-based clustering. Vertex-based clustering partitions the customer set directly, which is what the clustering step of cluster-first route-second does. The sweep, generalized-assignment, and spatial-temporal-demand procedures are all vertex-based in this sense. Route-based clustering instead requires a complete incumbent, grouping customers that already share a route or clustering the routes themselves by their barycenters.

Bent and Van Hentenryck (2010) study adaptive schemes that repeatedly decompose a Vehicle Routing Problem with Time Windows (VRPTW) instance from the current solution. In their reading, vehicle-based decompositions, which partition the vehicles of the incumbent across subproblems, cannot express temporal splits and may produce spatial splits that are not focused enough. They propose customer-based decompositions instead. On class R2 of the extended Solomon benchmarks, the customer-based variants improved on plain large neighborhood search where the vehicle-based ones did not.

Santini et al. (2023) compare decomposition strategies inside Adaptive Large Neighborhood Search (ALNS) (Pisinger and Ropke, 2007) and hybrid genetic search (Vidal et al., 2012) on the X instances. Route-based decompositions, which group customers that already share a route, outperformed path-based coarsening, which merges customers into supernodes. Barycenter clustering was the strongest strategy of those tested, improving both host algorithms relative to runs without decomposition. The benefit depended on the strategy and on the instance, and a poor decomposition failed to help at all. Subproblem size and decomposition frequency both required calibration. Their ALNS learns destroy and repair weights within the run, but the decomposition method itself stays a fixed, calibrated choice.

The two studies favor different paradigms. Bent and Van Hentenryck (2010) find customer-based decompositions stronger on a class of VRPTW instances, while

Santini et al. (2023) obtain their best results with a route-based strategy on the CVRP. Because the studies differ in problem, host heuristic, and benchmark, the comparison does not isolate the cause.

POPMUSIC (Taillard and Voß, 2002) starts from a complete solution and optimizes sub-parts of it until a local optimum is reached. Queiroga et al. (2021) apply the idea to the CVRP, using a branch-cut-and-price algorithm as a heuristic on subproblems of typically 25 to 200 customers. The main experiments cover instances of 302 to 1,000 customers. Long runs started from strong incumbents or CVRPLib BKS values improved several best known solutions, including some on very large instances. The method needs a complete solution before it can cut, and it uses a fixed subproblem policy. Its exact subsolver is also not an interchangeable off-the-shelf heuristic in the sense of Section 2.4.

2.3.4 Repairing Decomposition Boundaries

Each subproblem of a decomposition can be solved well in isolation while the assembled solution still carries defects along the cuts. A partition drawn from coordinates or route centroids uses no information about which customers a vehicle will end up serving together. Routes on either side of a cut may therefore be cheaper when reconnected across it. The reviewed methods handle these seams in four ways.

The first is to redraw the cut. Iterative schemes decompose again from the current solution at regular intervals (Bent and Van Hentenryck, 2010; Santini et al., 2023), so that a seam of one iteration can fall inside a subproblem of a later one. Whether any given seam is repaired then depends on where the next partition happens to cut.

The second is to take the subproblems from the incumbent in the first place. POPMUSIC follows this pattern (Taillard and Voß, 2002; Queiroga et al., 2021). Because every subproblem is carved out of the current solution, the method maintains no separate partition whose seams would need repair.

The third is an explicit improvement step after the subproblem solutions are concatenated. Kerscher and Minner (2025) apply local search to inter-route moves between subproblems, and prune it with relational data from the clustering step. A move is evaluated only when the two subproblems lie within a vicinity of each other under the clustering dissimilarity. Where fuzzy clustering was used, a customer additionally qualifies only when its highest membership degree falls below a threshold, which marks it as lying near a cluster border. The pruning therefore concentrates the improvement budget on the customers the decomposition placed at a perimeter.

The fourth is recombination. The set covering step of Kerscher et al. (2026) selects routes from a pool holding the output of many decompositions. A route that one partition cut badly can then be replaced by a route from another partition.

In all four, the partition information decides where the next subproblem falls or which moves the local search evaluates. We found no method in the reviewed literature that uses the partition boundaries to place a perturbation before a local search starts.

2.3.5 Adaptive Operator Selection and Algorithm Configuration

Every heuristic in this review exposes parameters, and many offer alternative operators. Eiben et al. (1999) distinguish parameter tuning, which fixes values before a run, from parameter control, which changes them during one. Offline configurators automate the tuning side. The irace package of López-Ibáñez et al. (2016) races candidate configurations on a set of training instances and eliminates those that a statistical test finds worse. The result is one configuration for the instance class that the training set represents.

Automated algorithm selection asks the same question at the level of whole algorithms. Kerschke et al. (2019) observe that for practically every intensely studied computational problem, different instances are best solved by different algorithms. Per-instance selectors exploit this complementarity by predicting from instance features which algorithm of a portfolio to run. The scale transition that Queiroga et al. (2026) observe on the CVRP is one instance of such complementarity. Both tuning and per-instance selection are trained before the test instance is solved.

Online operator selection adapts during the run instead. The ALNS of Ropke and Pisinger (2006) updates roulette-wheel weights of destroy and repair heuristics from scores collected during the search, and Pisinger and Ropke (2007) apply the same adaptive layer across five vehicle routing variants. Windras Mara et al. (2022) survey 252 publications on ALNS from 2006 to 2021, and Voigt (2025) reviews and ranks the operators used in ALNS for vehicle routing problems.

The selection problem can be read as a multi-armed bandit. The search observes the reward of the operator it applied, but not what the other operators would have produced (Auer et al., 2002). Fialho et al. (2010) analyze bandit-based adaptive operator selection mechanisms in this setting. Surveys of hyper-heuristics (Burke et al., 2013) and of machine learning in metaheuristics (Karimi-Mamaghan et al., 2022) place the work in a broader literature on choosing heuristics during a search.

The evidence on what adaptive selection buys is mixed. Turkeš et al. (2021) run a meta-analysis of 25 ALNS implementations against their non-adaptive variants. On average, the adaptive layer improved the objective function value by 0.14%, with a 95% confidence interval of 0.06 to 0.21%. Its value depends on the set of heuristics it selects from, and because the layer adds complexity, the authors recommend it only in specific situations.

Reinforcement learning replaces the roulette update with a trained policy. Kallestad et al. (2023) train a deep reinforcement learning agent that selects the next low-level

heuristic from features of the search state. It outperformed ALNS and uniform random selection on four combinatorial optimization problems, and, unlike ALNS, was not hurt by a larger set of heuristics. Reijnen et al. (2024) use deep reinforcement learning for the online control and configuration of ALNS. Both policies are trained on sets of instances before they are applied.

What this literature adapts are operators inside one search process. In the work reviewed here, neither the decomposition of the instance nor the choice of subsolver is among them. Online control also moves part of the design problem into the update rule. The adaptive layer of ALNS carries its own parameters, among them a reaction factor, a segment length, and the score values (Ropke and Pisinger, 2006).

2.3.6 Learned Decomposition

Machine learning for combinatorial optimization has grown into a research line of its own (Bengio et al., 2021). For routing, Kool et al. (2019) train attention-based construction policies with reinforcement learning on instances of up to 100 customers. Queiroga et al. (2026) observe that machine-learning methods for the CVRP were rarely tested on standard benchmark sets, and often relied on ad hoc instance generators instead.

Learning-based work on large instances has turned to decomposition. Li et al. (2021) train a Transformer to choose which spatially local subproblem to delegate to a black-box subsolver, applying the policy iteratively on instances of roughly 500 to 3,000 customers. Hou et al. (2023) train a two-stage divide method that partitions a CVRP instance into traveling salesman subproblems, and report real-time solutions beyond 5,000 nodes. Ye et al. (2024) combine a learned global partition policy with local construction in GLOP.

These policies are trained before the test instance is solved. They can be applied iteratively at test time, but the mapping from instance features to a partition or a delegation is not updated from the costs observed on that instance.

Karaahmetoglu and Kim (2026) use a pretrained large language model in place of a trained policy. The model selects clustering, balancing, and refinement actions over a hierarchical cluster tree from partition diagnostics alone, without feedback from solution costs. The method builds one partition per run, then routes the clusters with the guided local search of OR-Tools. It addresses a heterogeneous-fleet variant, and the preprint reports no gaps on either the X or the XL set.

2.4 Algorithmic Frameworks

Sörensen (2015) distinguishes a metaheuristic as a problem-independent framework from a problem-specific implementation of its guidelines, and Sörensen et al. (2018)

keep that vocabulary. This thesis uses three terms. A framework is a reusable control structure that can wrap existing solvers. A solver is a concrete implementation that searches a routing instance, such as AILS-II, FILO, FILO2, or PyVRP. A heuristic is any procedure that returns a feasible solution without an optimality guarantee. Every solver named above is a heuristic, though not every heuristic is a framework.

Kerscher and Minner (2025) propose the Decompose-Route-Improve (DRI) framework for the VRPTW, built on three phases. The decomposition phase clusters customers under a spatial-temporal-demand dissimilarity, which scales the pairwise spatial distance by the scheduling flexibility and the capacity consumption of the customer pair. The partition therefore reflects the objective and the constraints rather than geography alone. The routing phase solves each resulting sub-VRPTW independently with an existing solver, PyVRP in their experiments. The improvement phase then applies the pruned local search of Section 2.3.4 to the concatenated result.

Two properties of the DRI matter for the comparison drawn in Section 2.5. First, the number of subproblems is not a handcrafted constant. Kerscher and Minner (2025) evaluate the silhouette coefficient of the clustering over a range of candidate counts and take the maximizing value, so that the granularity of the partition adapts to each instance. They report that this matches a perfect-information baseline closely on clustered instances, at a fraction of its runtime. Second, that criterion measures cluster compactness and separation rather than routing cost, and it is evaluated once before the routing phase begins. The clustering algorithm, the dissimilarity metric, the vicinity size, and the fuzzy threshold are all compared offline in a sensitivity analysis and then held fixed. Their computational study covers the Gehring and Homberger instances together with a new set of 20 very large-scale VRPTW instances of 3,000 to 30,000 customers. On that new set, the DRI reports lower costs than PyVRP on almost every instance.

Kerscher et al. (2026) extend the pattern to the Heterogeneous Vehicle Routing Problem (HVRP) in DRSCI, whose four steps give it its name. Each iteration decomposes the instance with k -means clustering, either over customer vertices or over the routes of the current best solution. Each cluster becomes a sub-instance by duplicating the depot and assigning a sub-fleet to it. The routing step then generates routes for each sub-instance through one of four strategies. These solve it as several independent homogeneous VRPs, as connected prize-collecting VRPs, through a giant-tour partitioning procedure, or directly with an HVRP solver. All generated routes enter a persistent pool, and a set covering model selects a subset from it. A savings-based heuristic then removes the customers that end up served more than once, before local search refines the result.

How DRSCI chooses its configuration is the part closest to this thesis, so it is worth stating precisely. The clustering paradigm is a coin flip in every iteration, with

vertex-based and route-based clustering equally likely. The number of clusters and the routing strategy follow a scheme that their hyperparameter tuning selected over a fully random alternative. Under that scheme, every routing strategy is applied exactly once at the current cluster count before the count advances. The candidate counts are drawn from powers of two bounded by the route count of a feasible solution. Once a solution exists, the candidate range narrows to those counts that contributed at least one route to the current best solution. DRSCI therefore does use within-run feedback on one dimension of its configuration, by restricting a range rather than by weighting options. Their study covers heterogeneous-fleet instances of 101 to 1,001 vertices, and reports 14 improved best known solutions on the XH dataset and 247 new ones on the FSMVRPTW dataset. The published experiments run single-threaded, though the authors note that route generation is parallelizable by design.

An unpublished project study adapted DRSCI to the CVRP and reported a mean gap of 0.53% to the initial XL BKS values (Altendeitering and Bachmann, 2026). That study drew its decomposition and routing configuration uniformly at random in every iteration.

2.5 Research Gap

No published decomposition framework maintains weights over its decomposition and subsolver choices and updates them from the solution costs observed on the instance it is solving. The reviewed methods settle that configuration in one of five ways:

- by a fixed rule or a calibrated default (Taillard, 1993; Queiroga et al., 2021; Santini et al., 2023),
- by a clustering-quality criterion evaluated once per instance, before routing begins (Kerscher and Minner, 2025),
- by a schedule that cycles the routing strategies and narrows the cluster-count range to values that contributed routes to the incumbent (Kerscher et al., 2026),
- by a policy trained before the test instance is solved (Li et al., 2021; Hou et al., 2023; Ye et al., 2024), or
- by a pretrained model that reads partition diagnostics without cost feedback (Karaahmetoglu and Kim, 2026).

DRSCI (Kerscher et al., 2026) is the closest of these, and the distance to it is narrow enough to state exactly. Its narrowing rule does react to the instance at hand, but it acts on the cluster count alone. It selects uniformly inside the narrowed range, and it credits neither the clustering paradigm nor the routing strategy with the quality of

what they produced. The DRI (Kerscher and Minner, 2025) adapts its granularity per instance as well, from a criterion that never observes a routing cost.

The evidence on whether closing that distance pays off points in two directions. Turkeš et al. (2021) find a small average benefit from the adaptive layer in ALNS, whereas trained controllers outperformed roulette-wheel selection on the problems where they were tested (Kallestad et al., 2023). Neither body of work adapts a decomposition or a subsolver, so whether online control helps at that level has not been tested.

Algorithm-agnostic solver plug-in, parallel subproblem solving, route-pool recombination by set covering or set partitioning, and evaluation on the XL set all fall within the scope of this thesis. Taken separately, none of them is the gap. Table 2.1 records which published methods combine which of them.

Table 2.1: Research Gap Matrix for Large-Scale Routing Methods.

Method	Problem	P	D	SC	Ag	Configuration	Largest n
Máximo et al. (2024)	CVRP	no	no ^a	no	–	fixed	30,000
Accorsi and Vigo (2024)	CVRP	no	no ^a	no	–	fixed	1,000,000
Rochat and Taillard (1995)	VRP	no	no	yes	–	fixed	199
Cavaliere et al. (2022)	CVRP	no	no	yes	–	fixed	30,000
Queiroga et al. (2021)	CVRP	~ ^b	yes	no	~	fixed	1,000
Santini et al. (2023)	CVRP	~	yes	no	–	calibrated	1,000
Li et al. (2021)	CVRP	~	yes	no	yes	trained offline	3,000
Karaahmetoglu and Kim (2026)	HVRP	yes	yes	no	~	pretrained	500,000
Kerscher and Minner (2025)	VRPTW	no ^c	yes	no	yes	silhouette per instance	30,000
Kerscher et al. (2026)	HVRP	no ^c	yes	yes	yes	scheduled, range narrowed	1,001
Altendeitering and Bachmann (2026)	CVRP	~	yes	yes	yes	random	10,000
This thesis	CVRP	yes	yes	yes	yes	learned online	10,000

P = parallel subproblem solving, D = customer-set decomposition, SC = route-pool set covering or set partitioning, and Ag = algorithm-agnostic solver plug-in. ~ marks a partial capability. Largest n is the largest customer count in the original computational study.

^a Localized neighborhoods through granular arcs and selective vertex caching, not customer-set decomposition (Toth and Vigo, 2003; Accorsi and Vigo, 2021).

^b Subproblems are reoptimized within an intensification loop rather than solved as a standing parallel layer.

^c The published experiments run on a single thread. Both papers note that the subproblems are independent and that route generation is parallelizable by design.

3 Framework

This chapter presents DRISPI, the framework this thesis develops for the CVRP at very-large scale (Section 2.1). DRISPI repeatedly executes a decompose-route-improve (DRI) loop. Each iteration first partitions the customer set into smaller subclusters and solves one routing subproblem per subcluster with an existing monolithic solver, then concatenates the resulting routes and revises them near the subcluster boundaries with a dedicated improvement stage, boundary-guided adaptive iterated local search (BG-AILS). Hierarchical adaptive operator selection (HAOS) draws the decomposition and routing configuration for every iteration from a set of weighted alternatives and updates those weights from the outcomes it observes, rather than fixing one configuration for the whole run. Every route the DRI loop produces enters a persistent route pool; at scheduled intervals, DRISPI steps back from the current incumbent and recombines the pool through a set covering or set partitioning (SC/SP) model, followed by one further improvement pass. The loop continues until a wall-clock time limit or a run of iterations without improvement is reached.

Figure 3.1 shows the resulting control flow for one instance. HAOS (Section 3.1) draws one configuration per iteration; decomposition (Section 3.2) partitions the customer set into k subclusters under that configuration; routing (Section 3.3) solves each subcluster independently and concatenates the routes; BG-AILS (Section 3.4) revises the routes near the subcluster boundaries. Both the routing stage and BG-AILS add their routes to the route pool, and either can become the new incumbent if it improves on the current one. At scheduled intervals, the same iteration also runs set covering or set partitioning (Section 3.5) over the pool accumulated so far, followed by one further improvement pass (Section 3.6) whose output feeds the pool again and can likewise become the new incumbent; on every other iteration, this pair of stages is skipped. The loop then either returns to a fresh HAOS draw or stops. Figure 3.1 shows this control flow as one sequential loop; Section 3.7 shows how DRISPI executes it with several stages running in parallel across iterations instead.

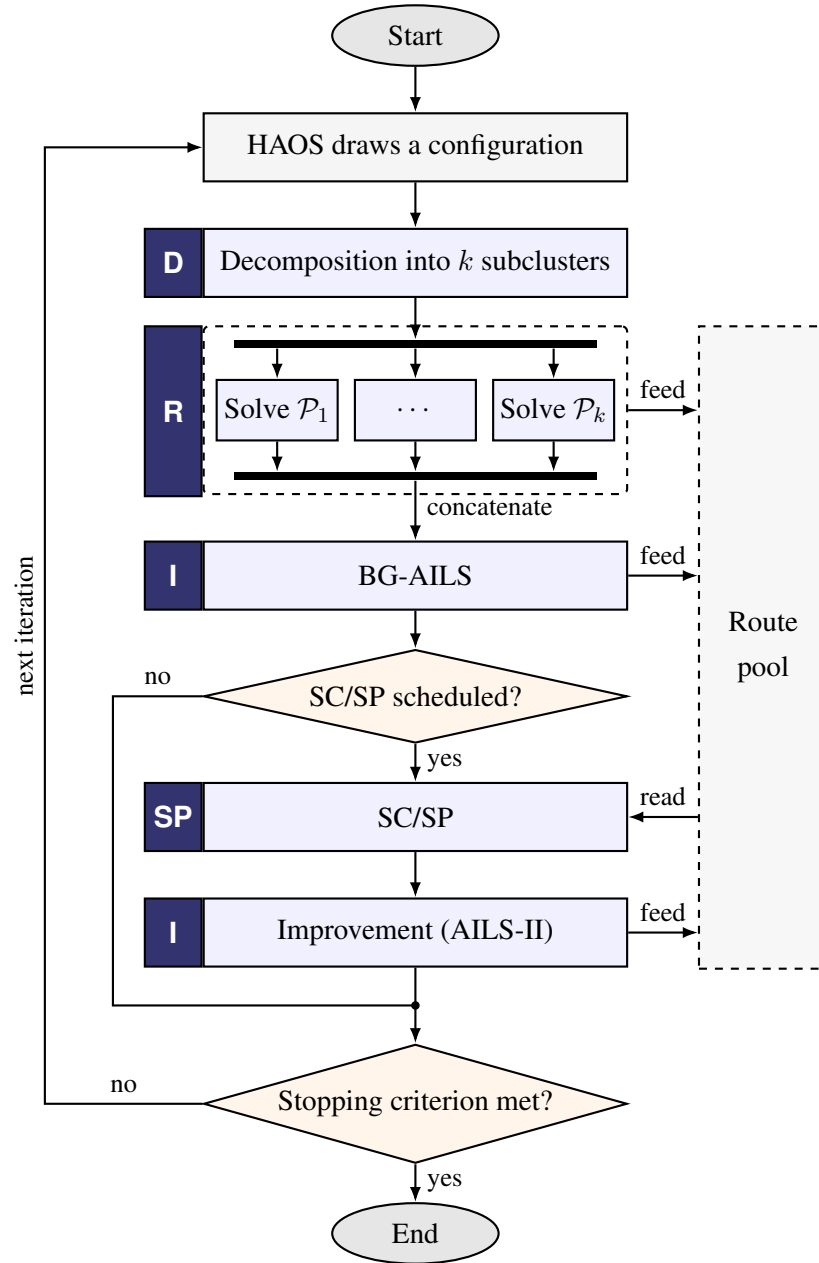


Figure 3.1: Sequential Flowchart of DRISPI.

The remaining sections address one stage each, in the order Figure 3.1 shows: Section 3.1 on HAOS, Section 3.2 on decomposition, Section 3.3 on routing, Section 3.4 on BG-AILS, Section 3.5 on the route pool and SC/SP, and Section 3.6 on the post-SC/SP improvement pass. Section 3.7 closes the chapter by assembling these stages into the parallelized pipeline DRISPI actually runs, together with the pseudocode for its control loop.

3.1 Hierarchical Adaptive Operator Selection

Ideally, a heuristic would always be tuned to no more than one specific problem instance. That would promise both, an effective and efficient approach configured to exploit that instance's size, spatial and demand distributions, and fleet capac-

ity. Recent benchmark sets however have deliberately combined a wide range of these characteristics to test whether performance transfers across instance structures (Uchoa et al., 2017; Queiroga et al., 2026). That is why finding and selecting a tailored configuration for every instance requires a costly tuning campaign. That is why most benchmarks are compromising on one fixed configuration for the entire problem set. This framework aims to forego this compromise through Hierarchical Adaptive Operator Selection (HAOS), an approach to adapt DRISPI’s configuration while solving the problem instead.

The principal methodological inspiration is adaptive large neighborhood search. Competing destroy and repair operators receive weights based on observed performance and are selected by roulette wheel (Ropke and Pisinger, 2006). The same adaptive layer has been applied across five vehicle routing variants (Pisinger and Ropke, 2007). HAOS adopts this principle but moves the adaptive choice outside a single local search. It instead controls the decomposition granularity, the representation and method used for clustering, the demand contribution cluster formation, as well as the subsolver. The adaptation is online because rewards update the weights while the current instance is being solved. The weights of each operator summarize observed performance on that instance. The Convergence to a globally optimal configuration is neither required nor guaranteed.

The operator choices can also be interpreted as arms of a multi-armed bandit. After an arm is drawn, the search observes a reward but does not know what the unselected arms would have produced. This analogy connects HAOS to the exploration–exploitation problem (Auer et al., 2002) and to bandit-based adaptive operator selection (Fialho et al., 2010). However, HAOS does not implement a confidence-bound policy, an explicit exploration bonus, or a separate exploratory action. Its roulette probabilities follow the current weights directly. Stochastic sampling leaves every positive-weight option selectable, while the probability-floor transformation described below raises normalized shares that become sufficiently small.

At the beginning of every DRI iteration HAOS draws one configuration across five levels. Level one selects the number of subclusters k from a scale-adaptive domain. Its base values are clipped to the instance’s minimum feasible fleet size and extended geometrically for larger instances. The number of arms is capped at ten to allow for convergence to occur within a reasonable wallclock time. Level two selects the demand weight λ_q from six evenly spaced values between zero and one; this value controls how strongly differences in customer demand influence the decomposition. Level three selects the clustering paradigm π : whether customers are clustered directly (the vertex-based paradigm) or through the routes of the current incumbent solution (the route-based paradigm). Level four selects a clustering method m from the wheel associated with π , with five vertex-based and three route-based methods in the deployed configuration. This conditional fourth level makes the selection hierarchical since the selection of m depends on π . Level five selects

Algorithm 1 Hierarchical Adaptive Operator Selection (HAOS)

input: wheels $W_1, W_2, W_3, W_4^v, W_4^r, W_5$; floors $p_{\min}^{(\ell)}$; decay γ ; base weight w_0 ; warm-up $i_0 = 10$

- 1: **function** DRAW(S^*)
- 2: draw k, λ_q, π, s from W_1, W_2, W_3, W_5 with probabilities (3.3)
- 3: **if** $\pi = \text{route}$ **and** S^* is undefined **then** $\pi \leftarrow \text{vertex}$
- 4: draw method m from W_4^π
- 5: $\theta \leftarrow (k, \lambda_q, \pi, m, s)$ $\triangleright$ credited
- 6: **if** $\pi = \text{route}$ **then** $k \leftarrow \min(k, |S^*|)$
- 7: **return** credited θ and applied $(k, \lambda_q, \pi, m, s)$
- 8: **end function**
- 9: **function** CREDIT(θ, r)
- 10: **if** $i \geq i_0$ **then** $w_a \leftarrow \max(w_a + r, w_0)$ for every arm $a \in \theta$ by (3.1)
- 11: **end function**
- 12: **function** DECAY()
- 13: **if** $i \geq i_0$ **then** $w_a \leftarrow \max(\gamma w_a, w_0)$ for every arm of every wheel by (3.2)
- 14: **end function**

one of four routing solvers s and applies it to every subcluster generated during the iteration. The first four levels therefore configure decomposition, and the fifth configures subproblem routing. Write $\theta = (k, \lambda_q, \pi, m, s)$ for this five-level configuration.

Two feasibility adjustments apply before the configuration is used; Algorithm 1 gives Draw, Credit, and Decay. Route-based clustering requires an incumbent solution. Without one, HAOS uses the vertex-based paradigm and redraws the method from the vertex wheel; these applied choices receive the subsequent credit. If k exceeds the number of incumbent routes under route-based clustering, the applied k is clipped to the route count. The immediate update retains the index of the k arm originally drawn, while route tags record the configuration that was applied. Each iteration is logged with the complete operator selection and the product of the five selected probabilities, which reports the joint probability of the realized configuration under the current wheels.

HAOS maintains six roulette wheels for the five levels, because of the conditional clustering method level. Each wheel tracks one weight per option. A reward r adds to the weight of every arm selected for the credited configuration. Once per iteration, every weight is also multiplied by a common decay factor. This uniform decay is the only temporal mechanism in the weights. Both operations retain the starting weight as a lower bound:

$$w_i \leftarrow \max(w_i + r, w_0), \quad (3.1)$$

$$w_i \leftarrow \max(\gamma w_i, w_0), \quad \gamma = 0.95 \text{ by default}, \quad (3.2)$$

with every wheel starting from the same weight $w_0 = 10.0$. Before an arm is drawn, HAOS first computes its normalized weight $\hat{p}_i = w_i / \sum_l w_l$. A level-specific floor is applied to these values, followed by renormalization:

$$p_i = \frac{\max(\hat{p}_i, p_{\min})}{\sum_j \max(\hat{p}_j, p_{\min})}. \quad (3.3)$$

The floor parameter is the minimum probability any weight of any wheel can decay to. For the five levels, the floors are 0.025 for k , 0.05 for λ_q , the clustering methods, and the solver, and 0.10 for the paradigm respectively. Because Equation 3.3 renormalizes after flooring, these values are nominal pre-normalization floors rather than strict lower bounds on the final probabilities. HAOS retains three elements of the adaptive layer proposed by Ropke and Pisinger (2006) and used for general vehicle routing by Pisinger and Ropke (2007): initially equal weights, quality-dependent scores, and roulette selection proportional to the resulting weights. Its additive reward, global decay, and six-wheel hierarchy are novelties introduced for DRISPI.

HAOS adapts its weights by receiving rewards from later stages of the current or following iterations. First, there is an immediate channel for results produced directly by the current DRI iteration and a deferred channel for routes that prove useful during later pool recombination. Table 3.1 gives the deployed reward values. Both channels compare a candidate with the incumbent best cost and the previous iteration’s cost, but the deferred scores are smaller because several earlier configurations can contribute routes to the same recombined solution.

Table 3.1: Default Immediate and Deferred HAOS Rewards.

Outcome	Immediate	Deferred
New incumbent best	10.0	7.0
Improvement over previous iteration only	4.0	3.0
No improvement	1.0	0.0
No solution	0.0	0.0

The immediate channel scores two candidates: the solution obtained after decomposition and subproblem routing, and the solution obtained after BG-AILS. Both rewards are credited to the same five-level configuration drawn for that iteration and therefore accumulate. Each selected arm receives the full reward. By doing so, HAOS learns from the performance of the complete configuration but does not isolate the contribution of one level or model interactions between levels. In the asynchronous pipeline of Section 3.7 the BG-AILS reward of iteration i is credited only after decompose-route of iteration $i+1$, so the next HAOS draw does not yet see it; both rewards still land on the θ of iteration i .

When route-based clustering draws a k larger than the incumbent route count, only

the applied k changes; the credited configuration θ keeps the originally drawn k -arm.

The deferred channel is evaluated after SC/SP has selected routes from the pool and the resulting solution has received its unrestricted improvement pass. Every stored route carries a tag containing the complete configuration under which it was generated. The deferred reward is credited once to every distinct tag among the routes selected by SC/SP, even if those tags originate from much earlier iterations. Routes created only during the unrestricted improvement pass are excluded because no HAOS selection produced them. Deferred attribution is necessary because the usefulness of one stored route may become visible only when it is combined with routes generated later.

During the first 10 iterations, HAOS performs the same five-level draw but suppresses both reward updates and decay. All weights therefore remain equal, making every option uniform within the wheel from which it is eligible to be drawn. This warm-up period lets the first incumbent and an initial route pool form before early outcomes influence later probabilities. Learning begins with iteration 10, after which the immediate and deferred updates follow the same weight system.

3.2 Decomposition

The decomposition layer runs at the beginning of every iteration in the DRI loop, adapting the decompose-route-improve terminology of Kerscher and Minner (2025), who propose the framework for large-scale vehicle routing problems with time windows that this work extends to the CVRP at very-large scale in a fully parallelized, self-tuning setting. HAOS (Section 3.1) controls this stage through four levels, redrawn at every iteration: first the number of subclusters k ; second the clustering paradigm, either vertex-based, which partitions the customers directly, or route-based, which partitions the routes of the current incumbent solution and propagates the resulting labels to their customers; third the clustering method used within that paradigm; and fourth the demand weight λ_q of the dissimilarity metric utilized a subset of clustering approaches. Additionally, a fifth quantity that also feeds the metric, the angular offset ϕ , is drawn uniformly from $[0, 2\pi)$ at every iteration independently of HAOS. ϕ ensures that the wraparound point described below does not stay fixed across iterations.

For the decomposition, we form a dissimilarity metric that combines three terms for every pair of customers i and j with coordinates (x_i, y_i) and (x_j, y_j) and demands q_i and q_j : their Euclidean distance, an angular term, and a demand term,

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + \lambda_\theta \Delta\theta_{ij}^2} \left(1 + \lambda_q \frac{|q_i - q_j|}{Q} \right). \quad (3.4)$$

$\Delta\theta_{ij}$ is the shortest angular distance between customer i 's and customer j 's polar

angle around the depot, computed with the offset ϕ introduced above. λ_θ scales the angular term to the average coordinate magnitude of the instance, so that neither term dominates across instances of different spatial extent. ϕ shifts where the $\Delta\theta_{ij}$ wraparound at 2π falls. The demand factor applies the multi-feature dissimilarity metric of Kerscher and Minner (2025), who combine spatial, temporal, and demand features, here to a setting without time windows, utilizing only the demand ratio $\lambda_q|q_i - q_j|/Q$ remains alongside spatial features.

Table 3.2: Inputs Used By Each Decomposition Method.

Method	Paradigm	Input	Angular	Demand
K-Means	vertex	raw coordinates	—	—
Fuzzy C-Means	vertex	raw coordinates	—	—
K-Medoids	vertex	dissimilarity matrix	✓	✓
Agglomerative (Average)	vertex	dissimilarity matrix	✓	✓
Agglomerative (Complete)	vertex	dissimilarity matrix	✓	✓
K-Means	route	route centroids	—	—
Agglomerative (Average)	route	route centroids	—	—
Agglomerative (Complete)	route	route centroids	—	—

Both clustering paradigms trace back to the classic cluster-first, route-second scheme for capacitated routing (Gillett and Miller, 1974; Fisher and Jaikumar, 1981), reviewed with route-first cluster-second and with vertex-based versus route-based clustering in Section 2.3.3. Vertex-based clustering partitions the customers directly and offers five methods (Table 3.2). K-Means (MacQueen, 1967) and Fuzzy C-Means (Bezdek et al., 1984) cluster on raw two-dimensional coordinates, Fuzzy C-Means hardened to a partition by assigning each customer to its highest-membership cluster. The remaining methods K-Medoids, partitioning around medoids (Kaufman and Rousseeuw, 1990), and Agglomerative Hierarchical Clustering under average or complete linkage (Kaufman and Rousseeuw, 1990) instead cluster directly on the dissimilarity matrix of Equation 3.4. They are the only three method arms that are influenced by λ_q and ϕ .

Route-based clustering represents each route by the centroid of its customers' coordinates and clusters the centroids with K-Means or with the same average or complete linkage, applied to the plain Euclidean distance between centroids rather than to the dissimilarity matrix (Table 3.2). This follows the barycenter-clustering decomposition strategy studied by Santini et al. (2023).

The clusters are then handed over in form of a set of independent subproblems to the following routing layer (Section 3.3) to be solved. The dissimilarity matrix, rebuilt from (λ_q, ϕ) , still feeds the boundary detection used at the end of the DRI loop in the BG-AILS (Section 3.4) stage regardless of which paradigm decomposition

uses. Labels then propagate from routes to customers directly, since every customer belongs to exactly one route in the incumbent solution.

3.3 Routing

Once decomposition produces the k subclusters (Section 3.2), the routing stage builds one sub-instance per subcluster, each with its own internal node IDs and its own copy of the depot. It then solves all k of them in parallel, independently of each other. The fifth and last HAOS level draws the solver once per iteration and applies it for every subcluster that iteration, so all k sub-instances are solved by the same one of four solvers. Each sub-instance receives a time budget of $\max(\text{floor}, \text{size} \times \text{rate})$ seconds, with a default rate of 0.06 seconds per customer and a floor of 5 seconds, so small subclusters are not starved of search time. The k sub-instances are dispatched to a pool of worker processes pinned to the cores reserved for this stage; if k exceeds the worker count, the remainder queue in further waves. Section 3.7 details how this pool fits into the wider core split. A wall-clock timeout derived from a fitted wave-time model guards against a hung worker.

The four solvers are monolithic, full-instance CVRP heuristics that DRISPI treats as black boxes: AILS-II (Máximo et al., 2024)¹, an adaptive iterated local search with an elite-solution pool for large instances that extends an earlier adaptive ILS hybridized with path-relinking (Máximo and Nascimento, 2021), runs as a Java subprocess; FILO (Accorsi and Vigo, 2021)², a ruin-and-recreate local search built on the COBRA granular-neighborhood library, runs as a compiled C++ subprocess; FILO2 (Accorsi and Vigo, 2024)³, an evolution of FILO for instances up to one million customers, also a C++ subprocess; and PyVRP (Wouda et al., 2024)⁴, a Python package implementing hybrid genetic search (Vidal et al., 2012; Vidal, 2022) that DRISPI calls in-process rather than as a subprocess. The four solvers selected are not an arbitrary shortlist. Queiroga et al. (2026), who introduce the XL instance set this thesis focuses on (Section 2.1), benchmark them against several other state-of-the-art heuristics and report that AILS-II and FILO2 dominate the best-known solutions on the XL range itself, while HGS-CVRP remains the strongest performer on the smaller, medium-scale instances that a typical DRISPI subcluster is closer in size to. The selection of solver level spans exactly this cluster size spectrum.

Once the routes returned by the k individually solved subclusters are concatenated, they form one combined solution for the full instance. Because the subclusters partition the customers, no route returned at this stage spans two subclusters, and the concatenation needs no reconciliation. Each route is tagged with the subcluster of its customers, information that BG-AILS (Section 3.4) reuses once its perturbations

¹<https://github.com/INFORMSJoC/2023.0106>

²<https://github.com/acco93/filo>

³<https://github.com/acco93/filo2>

⁴<https://pyvrp.org>

start moving customers across subcluster boundaries. This combined, subcluster-tagged solution is the input BG-AILS perturbs and re-optimizes next.

3.4 Boundary-Guided Adaptive Iterated Local Search

The decomposition introduces structural local optima by drawing partitions that are, from the routing problem’s point of view, potentially arbitrary: a cluster boundary sits wherever the drawn clustering method happened to cut, not necessarily wherever the customer geometry would suggest a natural route boundary. Each subcluster can be routed to a good local optimum in isolation (Section 3.3), yet the concatenated solution can still carry structural defects that only exist across the concatenation seams: routes on either side of a boundary that would be cheaper reconnected across it, or customers left on the wrong side of an arbitrary cut. A plain, unrestricted improvement pass over the whole solution could in principle fix these defects, but under a short iteration time budget it spends most of that budget on customers that already sit comfortably inside one subcluster’s interior, where the decomposition did nothing that needs correcting. Boundary-Guided Adaptive Iterated Local Search (BG-AILS) is DRISPI’s answer to this asymmetry: an improvement stage that reuses AILS-II’s own local search but spends its comparatively short time budget disproportionately on the boundary regions the decomposition itself created.

At its core, BG-AILS is AILS-II (Máximo et al., 2024), the same monolithic solver DRISPI can also draw as a HAOS routing arm (Section 3.3), run here purely as an improvement operator rather than a cold-start solver. Two additions to stock AILS-II, both introduced for this thesis, make that role possible: a `-initialSolution` flag that seeds the search with the combined post-decompose-route solution instead of building one from scratch, and a `-initialOmega` flag that overrides the starting value of ω , AILS-II’s own diversification-control parameter governing how strongly its internal perturbation cycle disturbs the incumbent before every local-search pass (Máximo and Nascimento, 2021). Seeding AILS-II with the concatenated solution is not sufficient on its own, however: a local search that starts from that solution and only ever considers its immediate neighborhood tends to repair it straight back toward the same seam defects the decomposition introduced, because nothing has told it that the seams, rather than a subcluster’s interior, are where a profitable move is more likely to be waiting. BG-AILS therefore precedes the AILS-II call with a purpose-built perturbation step, implemented in Python, that displaces customers specifically across the boundaries between subclusters, using a single, large-neighborhood move rather than the small single-customer or single-edge moves a subsequent local search pass could trivially undo in one or two steps. This mirrors the classical iterated-local-search argument, introduced for the double-bridge move on the traveling salesman problem (Martin et al., 1991) and generalized

since (Lourenço et al., 2003), that perturbation strength must be calibrated so that the following local search neither collapses straight back into the same local optimum nor is reduced to an effective random restart. No formal proof establishes this property for an arbitrary local search and perturbation pair; the argument, for the double bridge as for cross-reconnect, is structural rather than a theorem: both disconnect the current solution through a simultaneous multi-edge change that a local search built from smaller, incremental moves does not consider as a single step, and so is unlikely to reverse in one move of its own.

The perturbation step reuses the dissimilarity matrix built for the decomposition stage (Section 3.2) rather than defining a separate notion of distance, so that “boundary” means the same thing to BG-AILS as it did to whichever clustering method drew the partition. The deployed worker rebuilds that matrix from the same (λ_q, ϕ) rather than receiving d_{ij} . Algorithm 2 gives the full perturbation procedure, from boundary scoring through the chain of cross-reconnects, ending at the AILS-II call itself. For every customer i with cluster label $\kappa(i)$ under the current partition, BG-AILS first scores how close i sits to a customer outside its own cluster,

$$b_i = 1 - \frac{\min_{j: \kappa(j) \neq \kappa(i)} d_{ij} - d_{\min}}{d_{\max} - d_{\min}}, \quad (3.5)$$

$$\tilde{b}_i = \frac{b_i}{|\kappa(i)|} \left(1 + \alpha \max \left(0, \frac{\sigma - |\kappa(i)|}{\sigma} \right) \right), \quad (3.6)$$

where $d_{\min}$ and $d_{\max}$ are the smallest and largest nearest-other-cluster dissimilarity across all customers, so a customer with a nearby neighbor in a different cluster scores close to one and a customer deep inside its cluster scores close to zero. Equation 3.6 divides by the size $|\kappa(i)|$ of i ’s own cluster, so boundary attention is not monopolized by whichever cluster happens to contribute the most customers near a seam, and multiplies by a boost that grows as the cluster shrinks below a small-cluster cap σ (20 customers by default) up to a factor of $1 + \alpha$ ($\alpha = 0.5$ by default), so the very small clusters route-based decomposition can produce are not perturbed less just because they contribute few customers in absolute terms. After renormalizing $\tilde{b}_i$ to $[0, 1]$ via min–max to obtain $\hat{b}_i$, a boundary threshold τ gates the result,

$$\rho_i = \frac{\max(0, \hat{b}_i - \tau)}{1 - \tau}, \quad \tau = 0.5 \text{ by default}, \quad (3.7)$$

setting ranks at or below τ to zero and rescaling the remainder linearly back onto $[0, 1]$, structurally the same floor-then-renormalize pattern HAOS applies to its wheel probabilities (Equation 3.3), just against a fixed threshold rather than a per-level minimum share. This is the sense in which BG-AILS is more efficient than an unrestricted improvement pass under the same time budget: with $\tau = 0.5$ it never spends perturbation effort on the lower half of customers by boundary rank, in particular never on a customer sitting deep inside a cluster’s interior, and instead concentrates entirely on the customers the decomposition itself flags as most likely to be sitting

on a defect it created.

Boundary ranks are defined per customer, but the only perturbation BG-AILS applies acts on whole routes at once: a cross-reconnect move that cuts two routes at a random position each and exchanges their tails, so routes r_i and r_j cut after position a and b respectively become $r_i[:a] \frown r_j[b:]$ and $r_j[:b] \frown r_i[a:]$. This is the only perturbation operator BG-AILS implements. It is a large-neighborhood move by construction, since it can reassign an arbitrarily long tail of customers between two routes in a single step, which is precisely what keeps it from being trivially undone by the small single-customer and single-edge moves of a subsequent local search pass. A route's own weight for perturbation is the maximum boundary rank among its customers, and the first route of a cross-reconnect pair is drawn from these weights, either by roulette-wheel sampling or, under DRISPI's default greedy pair-selection setting, by taking the arg-max outright. Its partner is not drawn from the same weights again but from a per-route affinity toward each other cluster – the inverse of the mean dissimilarity between that route's customers and a given cluster's customers, row-normalized to sum to one – used first to pick which neighboring cluster to reach into from the first route's own affinity row, then to pick, among the routes majority-assigned to that cluster, the one with the highest affinity back toward the first route's own cluster. This two-step draw is what keeps a cross-reconnect pair spatially adjacent across one real cluster boundary, rather than pairing two routes that both happen to contain a boundary customer but face away from each other. BG-AILS repeats this draw-and-reconnect step for a chain of k iterations by default, one per subcluster (a $k - 1$ variant is also implemented but is not the deployed default), with each step in the chain acting on the routes as already modified by the previous one. When decomposition realizes only a single cluster, there is no boundary to guide perturbation toward, and BG-AILS skips the perturbation step entirely, falling back to a plain AILS-II improvement pass on the unperturbed combined solution.

The partner of the greedy first route is drawn in two steps from that route's affinity row: first a neighboring cluster, then the route inside it with the highest affinity back. The chain compounds because the route weights are recomputed after every reconnect, so each of the k steps can act on a route an earlier step already changed.

Figure 3.2 walks through one such call on XL-n1281-k29. Panel (a) is the concatenated decompose-route input, customers colored by the six subclusters of the pinned K-Medoids partition. Panel (b) maps pre-threshold boundary rank $\hat{b}_i$ and highlights the six route pairs the chain selects. Panel (c) shows those routes after the cross-reconnect cuts. Panel (d) is the solution after the AILS-II pass, customers colored by evaluated-move count from a separate instrumented run under the same input solution and budget.

Algorithm 2 Boundary-Guided Adaptive Iterated Local Search (BG-AILS)**input:** solution S ; labels $\kappa(\cdot)$; λ_q ; ϕ ; subcluster count k ; budget T **parameters:** σ , α , τ , ω_0 (Table 4.1)

- 1: **if** $k > 1$ **then**
- 2: rebuild d_{ij} from (λ_q, ϕ)
- 3: compute $\tilde{b}_i$ by (3.5)–(3.6) and ρ_i by (3.7) for every customer i
- 4: $\rho(r) \leftarrow \max_{i \in r} \rho_i$ for every route $r \in S$
- 5: **for** $t \leftarrow 1$ **to** k **do**
- 6: $r_a \leftarrow \arg \max_{r \in S} \rho(r)$
- 7: draw a neighboring cluster c from the affinity row A_{r_a} ,
- 8: $r_b \leftarrow \arg \max \{A_{r, \kappa(r_a)} : r \in S \text{ assigned to } c\}$
- 9: cut r_a and r_b at random positions p_a, p_b and exchange their tails
- 10: update $\rho(r_a)$ and $\rho(r_b)$
- 11: **end for**
- 12: **end if**
- 13: **return** AILS-II(S, ω_0, T)

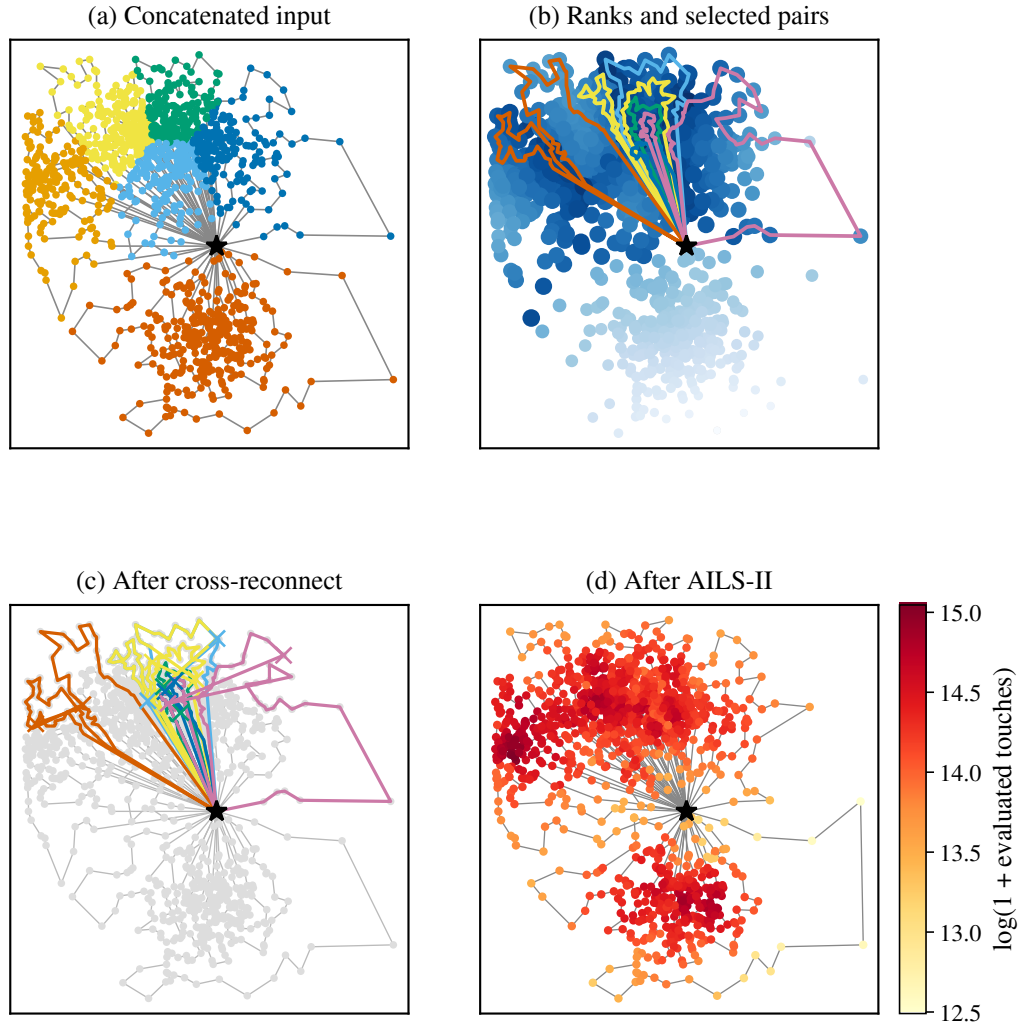


Figure 3.2: Illustrative BG-AILS Call on Instance XL-n1281-k29.

Colorbar Applies to Panel (d) Only.

Table 4.1 lists the values of the parameters this stage introduces on top of stock AILS-II. Of these, only the initial-omega override reaches into AILS-II itself; every other AILS-II hyperparameter governing its own local search and diversification control (the parameters $d_{\min}$, $d_{\max}$, γ , and φ of Máximo and Nascimento (2021)) is left at AILS-II’s own defaults. Overriding ω to 10 against AILS-II’s own starting value of roughly 30 lowers rather than raises the solver’s internal perturbation degree: BG-AILS has already displaced customers across subcluster boundaries with its own cross-reconnect chain before AILS-II starts, so the pass that follows is asked to intensify around that disruption rather than to add more of its own. The remaining parameters – the boundary threshold, the small-cluster cap and boost, the chain-count mode, and the pair-selection rule – are new and govern only the Python-side perturbation step described above.

The key difference between BG-AILS and a plain AILS-II improvement pass is therefore not the solver itself but this focus on boundary regions, imposed both by the perturbation step and by the short wall-clock budget BG-AILS is given. That budget follows neither the fixed per-customer rate used for subcluster solving (Section 3.3) nor a plain function of instance size, but a fitted model of the surrounding decompose-route stage’s own wall-clock time,

$$\text{budget} = \max(60\text{s}, 1.0 \times \hat{T}_{\text{DR}}), \quad (3.8)$$

$$\hat{T}_{\text{DR}} = 0.976 \times n_{\text{waves}}^{-0.180} \times \sum_{\text{waves}} \max(\text{per-cluster budgets in wave}), \quad (3.9)$$

refit on 835 post-strip pilot DRI iterations, where waves pack the current iteration’s per-subcluster solver budgets (Section 3.3) into the granted DRI worker count in partition-submit order. The campaign of Chapter 4 runs BG-AILS asynchronously under this budget: a dedicated worker perturbs and improves the current iteration’s solution while the main process decomposes and routes the next one. Section 3.7 returns to Equation 3.8 as the quantity that keeps that handoff tight.

Under this comparatively short wall-clock budget, BG-AILS should outperform an unguided AILS-II pass over the same combined solution, because the unguided pass spends an unknown and likely large share of that budget searching subcluster interiors the decomposition had already solved well. We tested this on all 100 XL instances (Section 4.1.1) at three checkpointed decompose-route solutions each, all arms forked from the same checkpoint: a stock AILS-II pass at the solver’s own defaults, a blind control that applies one cross-reconnect per subcluster to uniformly drawn route pairs, and BG-AILS in its deployed configuration (greedy first-route selection and no local-search mask), the control and BG-AILS at $\omega = 10$. Every arm receives the same AILS-II budget, the checkpoint’s own Equation 3.8 value; the Python-side perturbation of the two perturbing arms is a surplus on top of it, a median 0.28% of that budget for BG-AILS and 0.010% for the control. The checkpoints are generated under one pinned decomposition configuration, vertex

K-Medoids, with HAOS, the route pool, and SC/SP out of the loop, so what is compared is one improvement stage against another on identical input, not two full runs.

BG-AILS statistically significantly outperforms both baselines (paired Wilcoxon signed-rank tests, alternative greater, Holm-corrected in a family of two). It improves on the stock AILS-II pass by 0.056 pp of cost improvement over the input solution (Hodges–Lehmann paired difference, 95% CI [0.049, 0.063], Holm-corrected $p = 7.0 \times 10^{-11}$, ahead on 85 of 100 instances) and on the blind control by 0.058 pp (95% CI [0.052, 0.065], Holm-corrected $p = 2.7 \times 10^{-12}$, ahead on 89 of 100). The AILS-II comparison is the practical one and carries the perturbation together with the ω override as the package the framework deploys. The control comparison holds ω and chain length fixed and isolates the boundary guidance itself. The paired difference between the control and stock AILS-II is consistent with zero (Hodges–Lehmann -0.004 pp, 95% CI $[-0.011, 0.002]$), so the gain is not explained by perturbing the concatenated solution at all. Figure 3.3 shows the arm means and the paired differences behind those tests.

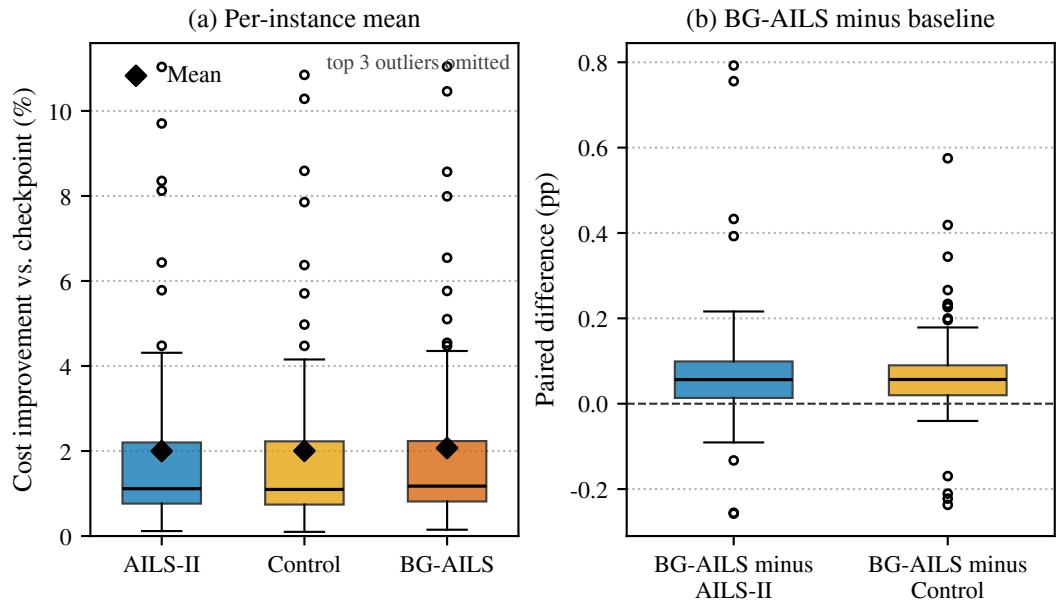


Figure 3.3: Per-Instance Mean Cost Improvement (a) and Paired BG-AILS Differences (b).

Three limits apply. The comparison is one DRI iteration, so it says nothing about what these routes are worth once the pool recombines them. It uses one pinned decomposition configuration, under which the clustering and the boundary metric read the same dissimilarity matrix, as they do for three of the eight deployed clustering methods but not for the five that cluster on raw coordinates or route centroids. The touch counts in Figure 3.2(d) come from a separate, identically configured instrumented run rather than from the runs the costs above are measured on.

BG-AILS closes the DRI loop: its output routes are contributed to the route pool like every other stage’s output (Section 3.5), and the next iteration begins with a fresh

HAOS draw over the decomposition and routing levels (Section 3.1). Periodically, rather than continuing the DRI loop for another iteration, DRISPI instead steps back and recombines the pool accumulated across many such iterations through set covering or set partitioning, the subject of the next section.

3.5 Route Pool and Set Partitioning

Periodically, DRISPI steps back from the current incumbent and recombines every route accumulated so far into one new candidate solution, using set covering or set partitioning (SC/SP) as the recombination model. Alternating a metaheuristic with an exact set-partitioning solve in this way is an established matheuristic pattern (Jourdan et al., 2009). Cavaliere et al. (2022) apply essentially this pairing to large-scale CVRP instances directly, by alternating LKH-3 local search with a restricted set-partitioning solve. Doing so only pays off if a shared memory of routes exists for it to recombine, so this section first describes that memory, the route pool, before turning to the SP and SC models themselves and the hyperparameter that decides between them.

The route pool stores one entry per distinct customer set $C_r \subseteq C$, where C is the customer set of the instance (Section 2.1), keyed by the frozen customer set rather than by the ordered sequence of customers a route visits them in: two routes serving the same customers in a different order, or at a different cost, are treated as the same column, and the cheaper of the two overwrites the other. This column-oriented memory, populated online during the DRI loop and periodically recombined through a partitioning model, follows the adaptive-memory principle of Rochat and Taillard (1995), who introduce the route pool and its set-partitioning recombination for vehicle routing.

Two pathways feed the pool every iteration, both through the same contribute-before-adopt rule: a candidate solution is added to the pool before the pipeline decides whether to adopt it as the new incumbent, so a losing candidate still leaves its routes behind for later reuse. The DRI loop feeds the pool at least twice per iteration, once from its decompose-route stage (Section 3.3) and once from the following improvement stage (BG-AILS (Section 3.4)). The SC/SP step covered in this section feeds the pool only indirectly. Solving SC or SP selects among the customer sets already in the pool and creates no new ones, so its own output adds nothing. The post-SC/SP-improvement pass that polishes its result (Section 3.6) is what can move customers between routes and through that generate new columns. Every contributed route carries a tag recording the full HAOS selection that produced it, that is, k , the demand weight, the paradigm, the method, and the solver, with a separate flag marking routes from the post-SC/SP polish; the tag survives even once HAOS has moved on to a different selection, which is what later lets reward attribution and pool inspection trace a route back to its origin. Whenever a candidate is adopted

as the new incumbent, its routes, already in the pool by the rule above, are marked elite; elite routes are exempt from eviction, so the pool never loses the one partition it is guaranteed to already have, namely the incumbent's own.

Every SC/SP solve also updates two running scores on each pool entry: A quality and a diversity score. The quality score represents a column's usefulness for the solver during the SC/SP solve. Therefore, an entry never touched by a solve carries no quality score yet. The diversity score is the mean Jaccard distance (Jaccard, 1901) between a route's customer set and every other pool entry's customer set,

$$\bar{J}(r) = \frac{1}{|R| - 1} \sum_{r' \in R \setminus \{r\}} \frac{|C_r \Delta C_{r'}|}{|C_r \cup C_{r'}|}, \quad (3.10)$$

where R is the pool at the time of the update, recomputed for every entry whenever any solve happens. New quality or diversity scores are appended to a three-value window, from which final scores are read as the mean of their respective window. This ensures that single unusually high or low values do not dominate a route's standing.

The route pool has a maximum size (10,000 columns by default), to keep the computational complexity from blowing up. We therefore establish an eviction policy that ranks the non-elite entries on both scores and combines the ranks into one fitness value. Among the entries that have been scored at least once, ascending order by mean quality score gives rank one to the weakest route and the highest rank to the strongest; independently, ascending order by mean diversity score gives rank one to the most redundant route and the highest rank to the most unique. Writing $\rho_q(r)$ and $\rho_d(r)$ for these two ranks, fitness is $\Phi(r) = \rho_q(r) + w_d \cdot \rho_d(r)$, with the diversity weight w_d (0.5 by default) controlling how much redundancy counts against a route relative to raw quality. The entries with the lowest $\Phi(r)$ are evicted first. Because both terms grow toward the same end of their respective orderings, a route that the LP relies on heavily but that is also nearly identical to several other pool entries can rank as a stronger eviction candidate than a weak but singular one, because redundancy drives the diversity term up regardless of quality. The eviction policy only triggers once the pool exceeds its maximum size and then only removes the overflow amount. Unsolved entries are evicted only if the overflow cannot be covered by scored, non-elite entries alone. Elite entries are categorically exempt from eviction.

The recombination model itself treats the pool as a column set. Let R be the set of routes currently in the pool, c_r the cost of route $r \in R$, that is, the Euclidean depot-customers-depot round trip over the travel costs of Section 2.1, and $x_r \in \{0, 1\}$ a decision variable selecting route r . Set partitioning, in the formulation Balinski and

Quandt (1964) introduce for exactly this delivery-routing setting, reads

$$\min \sum_{r \in R} c_r x_r \quad (3.11)$$

$$\text{s.t.} \quad \sum_{r \in R: i \in C_r} x_r = 1, \quad \forall i \in C \quad (3.12)$$

$$x_r \in \{0, 1\}, \quad \forall r \in R. \quad (3.13)$$

Solving Equations 3.11–3.13 recombines route fragments that have never been jointly considered, for example because they may have entered the pool from different subclusters, different HAOS selections, or different iterations, into one coherent solution that still covers every customer exactly once. This is the sense in which SP looks across the boundaries any individual DRI iteration’s decomposition imposes, something no individual subcluster solve or BG-AILS perturbation can do on its own.

DRISPI builds this model once in Gurobi⁵ and solves it twice. It first relaxes Equation 3.13 to the continuous $x_r \in [0, 1]$ and solves the resulting linear program. This LP relaxation allows the solver to assign fractional weights rather than the binary ones to each route, making it significantly easier and faster to solve. The resulting fractional weights are then utilized as the quality scores fed back into the route pool above, so every SC/SP solve doubles as a scoring pass over the pool regardless of what it ultimately selects. This follows the same spirit in which column-generation approaches read pricing information out of LP duals (Lübbecke and Desrosiers, 2005), even though DRISPI’s route pool is populated by heuristic solves rather than a pricing subproblem. For the second solve, the variables are then switched to binary, warm-started from the LP solution, and re-optimized as a mixed-integer program under a wall-clock time limit and a relative optimality gap (720 seconds and 0.025% by default). A wallclock time limit is required to cut off the search with a good, if not provably optimal, solution rather than run to completion.

The equality constraint of Equation 3.12 is only a meaningful choice if the pool offers more than one real alternative for most customers. A pool still dominated by a single incumbent partition would just reproduce it again and again under SP. DRISPI therefore checks, before every SC/SP trigger, whether every customer in the instance is covered by at least a fixed minimum number of pool routes (five by default). Only when that holds does SP run. Otherwise DRISPI falls back to the less constrained set covering (SC), the inequality-constrained relaxation of the same formulation (Garfinkel and Nemhauser, 1969), whose formal problem statement and LP relaxation are given in Appendix 1. SC only requires every customer to be covered at least once rather than exactly once, a condition the pool always satisfies, because the elite routes described above are never evicted and already form a feasible partition on their own. This is also why the fallback is safe and not merely

⁵<https://www.gurobi.com>

convenient: SP is never blocked by infeasibility, only by a pool that has not yet accumulated enough alternatives, and SC exists to give the SC/SP step something well defined and always solvable to run on every scheduled trigger, warmup aside, no matter how much the pool has matured. The price of that permissiveness is that SC's selected routes can overlap on a customer, meaning a customer can be visited by more than a single route and therefore more than exactly once. Since this would violate our problem statement formulated in Section 2.1, a savings-based deduplication heuristic then removes each duplicated customer from every selected route except the one where removing it would save the least cost, restoring a feasible partition.

A solved partitioning problem will pair routes into a new candidate solution which haven't been evaluated together before. And while search heuristics have been employed in identifying the routes used, the resulting new set of routes has not been polished by evaluating for example inter-route moves amongst each other. To do just that the SC/SP step is combined with a standard-AILS pass (Section 3.6) to improve the partitioning result even further if possible.

3.6 Improvement

Once SC/SP (Section 3.5) recombines the route pool into one partition, DRISPI runs one further pass over it using AILS-II as solver, seeded only with that partition as an initial solution. Unlike in BG-AILS (Section 3.4), this step focuses less on perturbation but rather refinement by local search. This standard improvement step also allows the solver to touch all routes instead of biasing moves towards certain neighborhoods. The pass runs under its own wall-clock budget (360 seconds by default) on the same core reservation as the SC/SP solve that precedes it; Section 3.7 covers how exactly a parallel architecture of DRISPI looks.

The refined routes feed back into the route pool exactly like every other stage's output (see Section 3.5): a route whose customer set matches one SC/SP already selected keeps that route's existing tag, while a route AILS-II creates by moving customers between routes is added as new, tagged as an improvement route rather than credited to a HAOS selection of its own.

This is also where the deferred half of HAOS's reward system (Section 3.1) closes. The pass's outcome, a new best, an improvement over the previous iteration without beating the incumbent, or neither, is scored through the deferred branch of HAOS's reward function and credited not to the current iteration's own draw but to every distinct HAOS tag the routes SC/SP selected already carried, that is, to whichever earlier decompose-route or BG-AILS draws produced those specific routes, regardless of how many iterations ago that was. Improvement-route tags are excluded from this credit, since AILS-II created them without any HAOS draw of their own to reward.

3.7 Parallelized Pipeline Orchestration

The preceding sections describe HAOS, decomposition, routing, BG-AILS, the route pool and SC/SP, and the post-SC/SP improvement pass as if the DRI loop executed them one after another, the sequential picture Figure 3.1 shows. DRISPI instead splits eight cores into three disjoint groups: six for the main loop and the DRI routing pool of Section 3.3, one for the BG-AILS worker of Section 3.4, and one for the SC/SP and post-SC/SP improvement worker of Sections 3.5 and 3.6. The main process shares the six DRI cores rather than occupying a ninth.

Figure 3.4 shows this overlap across two consecutive iterations. After decompose-route of iteration i the loop waits for job $i-1$ and drains it ①, then launches job i on the new solution ②. After decompose-route of iteration $i+1$ the same pair of steps waits for job i , drains it, and launches job $i+1$. A finished SC/SP job is drained without blocking ③ at the start of the following iteration, before the next HAOS draw.

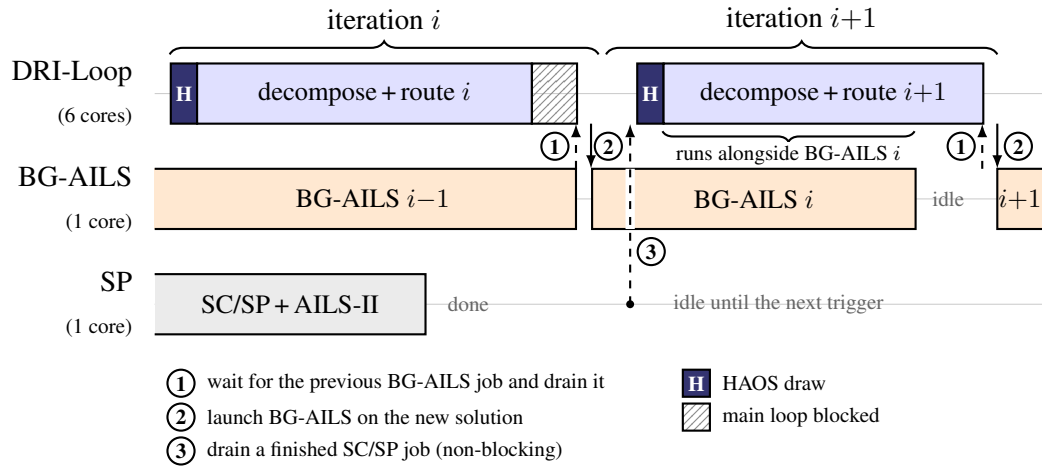


Figure 3.4: Two Consecutive DRISPI Iterations in One 8-Core Container (6 for DRI, 1 for BG-AILS, and 1 for SP). Block Lengths Are Illustrative.

Equation 3.8 sizes the BG-AILS budget to the decompose-route stage it overlaps with, which is what keeps the wait at ① short. A budget much shorter than the next decompose-route pass would leave the BG-AILS core idle before the handoff; a budget much longer would leave the DRI cores blocked on a job that already had the next iteration's input ready. In Figure 3.4 the wait after decompose-route of iteration $i+1$ vanishes because job i finished during that pass.

The SC/SP worker uses a skip policy rather than a blocking handoff. The campaign checks the trigger only at iteration boundaries, so a job starts at the first boundary after the 20-minute mark, not exactly at it. DRISPI then dispatches a snapshot of the route pool and the incumbent only if the worker is idle; if it is busy, the trigger is skipped rather than queued. The worker owns both the SC/SP solve and the post-SC/SP improvement pass of Section 3.6, running them one after another on the

same core. Those two ceilings, 720 s and 360 s, sum to 18 minutes and therefore leave two minutes of slack under the 20-minute interval; a skip is possible only if enqueue and overhead consume that slack.

Algorithm 3 is the deployed loop.

Algorithm 3 DRISPI Main Loop as Deployed (Async. BG-AILS and SP)

input: instance; time limit Γ ; stall limit Π
initialize: $R \leftarrow \emptyset$; S^* undefined; $i \leftarrow 0$

- 1: **while** elapsed time $< \Gamma$ **and** iterations without improvement $< \Pi$ **do**
- 2: **if** the SC/SP job has finished **then**
- 3: $\text{ABSORB}(S_{\text{SP}}, \Theta_{\text{SP}}, \text{deferred})$ $\triangleright \Theta_{\text{SP}}$: tags of selected routes
- 4: **end if**
- 5: $(\theta, \text{config}) \leftarrow \text{DRAW}(S^*)$ $\triangleright$ Algorithm 1
- 6: decompose into k subclusters under config and a fresh ϕ $\triangleright$ Section 3.2
- 7: **for** each subcluster **do in parallel**
- 8: route it with the drawn solver $\triangleright$ Section 3.3
- 9: **end for**
- 10: $S_{\text{DR}} \leftarrow$ concatenated subcluster routes; $\text{ABSORB}(S_{\text{DR}}, \{\theta\}, \text{immediate})$
- 11: **if** a BG-AILS job is in flight **then**
- 12: wait for its result (S_{BG}, θ') ; $\text{ABSORB}(S_{\text{BG}}, \{\theta'\}, \text{immediate})$
- 13: **end if**
- 14: dispatch BG-AILS on $(S_{\text{DR}}, \kappa, \lambda_q, \phi)$ $\triangleright$ Algorithm 2; θ for credit
- 15: **if** SC/SP is due **and** the sp worker is idle **then**
- 16: dispatch SC/SP-JOB(R, S^*) to the sp worker
- 17: **end if**
- 18: $\text{DECAY}()$; $i \leftarrow i + 1$
- 19: **end while**
- 20: **return** S^*
- 21: **function** $\text{ABSORB}(S, \Theta, \text{channel})$
- 22: $r \leftarrow$ reward of S in this channel (Table 3.1)
- 23: add the routes of S to R ; **if** $c(S) < c(S^*)$ **then** $S^* \leftarrow S$
- 24: **foreach** $\theta \in \Theta$ **do** $\text{CREDIT}(\theta, r)$
- 25: **end function**
- 26: **function** $\text{SC/SP-JOB}(R, S^*)$
- 27: **if** min. coverage of $R \geq 5$ **then** $S \leftarrow \text{SP}(R)$ **else** $S \leftarrow \text{dedup}(\text{SC}(R))$
- 28: **return** AILS-II(S) under the improvement budget $\triangleright$ Section 3.6
- 29: **end function**

After decompose-route the loop waits for the previous BG-AILS job ①, drains it through Absorb, and launches the next job on $(S_{\text{DR}}, \kappa, \lambda_q, \phi)$ ②, keeping θ only for later credit. A finished SC/SP job is absorbed without blocking ③ at the start of the next iteration. A synchronous mode that runs both stages inline is implemented as

a code option and recovers the sequential picture of Figure 3.1.

4 Computational Study

This chapter tests DRISPI on the 100 XL instances of Queiroga et al. (2026) against the eight monolithic solvers benchmarked there. The study aims to answer two questions: The first is whether an adaptive decomposition framework that learns its decomposition and routing choices during a run can match state-of-the-art monolithic solvers at this scale. The second is whether the weights HAOS learns along the way provide interpretable insights about the instances they were learned on.

Section 4.1 describes the dataset, specifies implementation and hardware specifications, the parameter values, and explains experimental design choices. Section 4.2 shows and analyzes the results. Here we answer the primary and secondary research questions through comparison with the published solvers, breaks the results down by instance characteristics, and examines the learned HAOS weights and the convergence of individual runs.

4.1 Experimental Design

All 100 instances are solved by the same DRISPI configuration. Nothing is tuned per instance by design without any exceptions. Section 4.1.1 describes the instances and the reference solutions used as baseline, section 4.1.2 the software and hardware the campaign ran on, section 4.1.3 the parameter values, and section 4.1.4 the number of seeds and the statistical precision it buys for our experiment.

4.1.1 Dataset

The XL set (section 2.1) extends the generation scheme of the X instances of Uchoa et al. (2017) to up to 10,000 customers. Every instance places its nodes on a $[0, 1000] \times [0, 1000]$ grid with Euclidean distances and combines one level of each of four attributes. The depot is Random, Central at (500, 500), or Eccentric at (0, 0) in the bottom left corner. Customers are Random, Clustered, or Random-Clustered around a small number of seed points. Demand follows one of seven distributions: unitary (U), four ranges of increasing width (1–10, 5–10, 1–100, 50–100), a quadrant-dependent range (Q), or many small and few large demands (SL). The target average route length r lies in one of seven bands, from ultra-short ($r \in [3, 5]$) to ultra-long ($r \in [50, 200]$), and capacity Q is set so that a solution with the minimum number of routes attains it. The instance name references n and that minimum route count, e.g. XL-n4535-k1134. This k equals $\lceil \sum_{i=1}^n q_i / Q \rceil$, making it the

mathematical minimum number of routes, because fewer vehicles could not carry the total customer demand under capacity set to Q . This k is fixed by the benchmark and unrelated to the subcluster count k of Section 3.1. Table A.1 breaks down the above mentioned characteristics on a per-instance basis along with DRISPI’s result and comparison to initial and current BKS values.

The initial BKS values of Queiroga et al. (2026) come from the solver runs described in section 2.2 and were released as baselines with the CVRPLib BKS Challenge, a community competition opened on January 12, 2026, in which any team can submit an improved solution for automatic verification. By the current record,⁶ the challenge has improved 99 of the 100 initial values, leaving only XL-n1094-k157 to still hold its initial value. We report the percentage gap to baselines such as the initial and the current BKS:

$$\text{Gap} = 100 \times \frac{c - c_{\text{BKS}}}{c_{\text{BKS}}}. \quad (4.1)$$

The quantity c is the reported solution cost. The current BKS represents the official best known solution. However, we primarily compare against initial BKS, because published solvers produced it in two-hour single-thread runs on specified hardware (Section 2.2), using us an experimental design we can reproduce to allow for any algorithmic comparability. The official BKS values on the other hand have been achieved without constraints regarding computational, with many of them coming from day-long runs that rely on raw compute, rendering any fair comparison infeasible.

4.1.2 Framework Implementation

All experiments in this chapter run on a single, shared department server under Ubuntu 22.04.5 LTS with two Intel Xeon Platinum 8280L CPUs at 2.70 GHz (28 cores each, 56 cores and 112 threads total) and 5,928 GiB of RAM, with no GPU. DRISPI runs inside Docker containers (Docker 28.1.1) pinned to a disjoint physical-core range with `-cpuset-cpus`. This pinned CPU count, not a library thread-pool default, sets the framework’s parallelism budget.

The campaign presented in section 4.2.1 is comprised of all 100 instances, each run on three seeds with a wallclock runtime budget and termination criterion of $T=2h$ (300 runs, 7,200 seconds each). For that we pin 48 of the server’s 56 physical cores as six exclusive 8-core Docker containers, three per socket, parallelizing as outlined in section 3.7 with a split into six DRI workers, one BG-AILS worker, and one SP worker. SMT siblings are left idle. Runs are packed into waves ordered by instance size so that the three seeds of the largest instance, XL-n10001-k1570, never share a wave. The full campaign takes 50 waves and about 100 hours of wall-clock time.

⁶https://galgos.inf.puc-rio.br/cvrplib/index.php/en/bks_challenge/score/instances, accessed September 7, 2026.

DRISPI uses Python 3.11, with dependencies locked via `uv`: Libraries PyVRP 0.13.3 (HGS-CVRP), scikit-learn 1.8.0 (K-Means and Agglomerative Clustering), scikit-learn-extra 0.3.0 (K-Medoids), scikit-fuzzy 0.5.0 (Fuzzy C-Means), gurobipy 13.0.1 (SC/SP), and `vrplib` 2.1.0 for instance parsing. AILS-II, FILO, and FILO2 are vendored as git submodules pinned to a fixed commit each and compiled at image build time, rather than resolved from a floating branch, so a rebuild reproduces the exact binaries behind the results in Section 4.2.1. The whole framework runs inside a Docker image built from this locked environment, so the software stack behind every reported run is fixed independently of the host it executes on.

4.1.3 Hyperparameters

Table 4.1: Hyperparameters of the Benchmark Configuration.

Parameter	Value	Defined in
<i>HAOS</i>		
Warm-up iterations	10	Section 3.1
Decay γ	0.95	Equation 3.2
Starting weight w_0	10.0	Equation 3.1
Floor $p_{\min}$: k / λ_q , method, solver / paradigm	0.025 / 0.05 / 0.10	Equation 3.3
Immediate and deferred rewards	Table 3.1	Section 3.1
<i>Decomposition and routing</i>		
Base k arms	{2, 3, 4, 6, 8, 10, 12}	Section 3.1
Maximum number of k arms	10	Section 3.1
Minimum routes per subcluster	8	Section 3.1
Minimum spacing between k arms	2	Section 3.1
Subcluster time budget	$\max(5 \text{ s}, 0.06 \text{ s} \cdot n_c)$	Section 3.3
<i>BG-AILS</i>		
Boundary threshold τ	0.5	Equation 3.7
Small-cluster cap σ / boost α	20 / 0.5	Equation 3.6
Initial ω_0	10	Section 3.4
First-route selection	greedy, unique first	Section 3.4
Chain length	k	Section 3.4
Boundary mask on first local search	on	Section 3.4
Budget floor / margin	60 s / 1.0	Equation 3.8
<i>Route pool, SC/SP, and improvement</i>		
Maximum pool size	10,000	Section 3.5
Diversity weight w_d	0.5	Section 3.5
SC/SP trigger interval	20 min	Section 3.7
Minimum coverage for SP	5	Section 3.5
MIP time limit / relative gap	720 s / 0.025 %	Section 3.5
Improvement pass time limit	360 s	Section 3.6

Table 4.1 lists the parameter values of the benchmark configuration, which all 100 instances and all three seeds share. Chapter 3 introduces each parameter with its mechanism, the table collects the values used to achieve the results presented in section 4.2. Most parameters were tuned during development in isolated pilot studies per category (HAOS, Decomposition and Routing, BG-AILS, Route Pool, SC/SP,

and Improvement). These small studies were comprised of a few instances and seeds, usually running a limited parameter grid search, aimed at finding a reasonable default rather than an optimum, and these studies are not reported here. The only exception is the BG-AILS comparison of Section 3.4, which ran on all 100 XL instances and fixed the first-route rule and the boundary mask; these two settings are therefore chosen on the benchmark set itself. No sensitivity analysis over the full set was run for any of the other values.

Not included in the table are the option sets HAOS draws from: the number of subclusters k , the six values of λ_q , the two paradigms, the eight clustering methods of Table 3.2, and the four routing solvers of Section 3.3. They define the search space of HAOS rather than a value within it. Also excluded are settings regarding the campaign architecture and experimental design: the core split and asynchronous modes of Section 3.7, the fitted coefficients of the BG-AILS wall-time model (Equation 3.9), and the time limit and seeds that Section 4.1.4 covers.

4.1.4 Replication Design

The campaign runs DRISPI on every one of the 100 XL instances three times, at seeds 11, 22, and 33. The seed drives the weighted HAOS draws, the angular offset, and the random choices inside decomposition. It is also passed to PyVRP, FILO, and FILO2, but AILS-II has no seed flag. This makes the routing phase, BG-AILS and the post-SC/SP improvement pass independent of the seed. This makes the runs non-reproducible. Additionally, the plugged-in solvers run under wall-clock limits, so how far their search gets depends on machine load, and the worker pool schedules subcluster jobs in no fixed order. This affects again, the entire routing phase, BG-AILS, and the post-SC/SP improvement pass alike, so two runs at the same seed can differ in solution quality. Disclosing the seeds therefore does not suffice for exact reproducibility. The three runs per instance are exchangeable replications.

Because each run is stopped after 7,200 seconds (120 minutes) of wall-clock time rather than at convergence, each run resembles a terminating simulation in the sense of Law and Kelton (2000): a fixed initial condition, the first DRI iteration's decomposition and routing pass (Section 3.2), and a fixed stopping rule rather than a steady state. The replication method of Law and Kelton (2000, ch. 9) therefore applies.

The precision target refers to the fixed set of 100 instances, which are a design factor (Queiroga et al., 2026) rather than a sample from a larger population, so the seed is the only randomness the design controls. Using $\bar{G}$ the grand mean gap and $\bar{\sigma}^2$ the average per-instance seed variance, we get $\text{Var}(\bar{G}) = \bar{\sigma}^2 / (I \cdot S)$ with $I = 100$. The $I \cdot S$ observations are independent by construction. Every run is a separate process with its own random-number stream, and no pool, cache, or counter is shared across runs (Chapter 3). This interval does not extend to the published baselines of Section 4.2.1, which report one value per instance and no

variance.

A pilot campaign of three diverse instances with four seeds each measured a root-mean-square per-instance seed standard deviation of 0.118 pp at the 7,200-second mark. The target half-width was fixed in advance at 0.03 pp, small enough to resolve the 0.02–0.06 pp effects seen in earlier component tests. Table 4.2 forecasts the standard error and the 95% half-width of $\bar{G}$ at the pilot value. Every S from one to four meets the target. This would in fact still hold true at a 99% interval, but precision alone does not decide S .

Table 4.2: Forecast Standard Error and 95% Half-Width of $\bar{G}$ by Seed Count S .

S	SE (pp)	Half-width (pp)
1	0.0118	0.023
2	0.0083	0.016
3	0.0068	0.013
4	0.0059	0.012

Three other considerations set $S = 3$. At $S = 1$ there are no within-instance degrees of freedom and no interval at all. At $S = 2$ a discrepancy between two runs is visible, but cannot be attributed to either of them. At $S = 3$, however, one anomalous run stands out against the other two. We also require $S > 1$ to present a best-of- S column, which is the convention on this instance set (Queiroga et al., 2026).

A fourth seed would add 100 runs and 1,600 core-hours to shrink the half-width from 0.013 to 0.012 pp. That gain is too small next to a second source of error. Two pilot campaigns that differed only in execution environment produced means 0.039 pp apart, three times the $S = 3$ half-width. Such a batch effect shifts every run of a campaign alike, so more seeds do not reduce it, and the budget of a fourth seed would not buy a meaningfully more precise $\bar{G}$ (Barr et al., 1995; Johnson, 2002). Neyman allocation, which assigns seeds in proportion to each instance’s standard deviation (Neyman, 1934; Cochran, 1977), would have further complicated the experimental design while only cutting the standard error of $\bar{G}$ by 8.8% at the same total budget. Therefore, every instance receives three seeds.

The pilot value was conservative. In the campaign, the mean per-instance seed standard deviation is 0.035 pp (Section 4.2.1), about a third of the pilot value, which puts the realized 95% half-width of $\bar{G}$ near 0.004 pp. This interval applies to the mean over seeds only. Best-of- S is an extreme-value statistic whose expectation depends on S , so it is not comparable across studies with different replication counts. Section 4.2.1 uses it descriptively and focuses its main comparison and the answer to our primary research questions on the mean-of- S gap.

4.2 Results

Short introduction.

4.2.1 Performance Comparison against State-of-the-Art Monolithic Solvers

Table 4.3 compares DRISPI against the eight monolithic solvers Queiroga et al. (2026) benchmark on the 100 XL instances: AILS-II, FILO, FILO2, KGLSXXL, HGS-CVRP, SISRs, LKH-3, and OR-Tools. For every solver we recompute the percentage gap to the current official BKS, the record listed by the CVRPLib BKS Challenge (Section 4.1.1), for both the mean-of- N and best-of- N cost the original paper reports ($N = 60$ for the published solvers, $N = 3$ for DRISPI), then average that per-instance gap over all 100 instances and, separately, over the fifty instances with fewer than 3,400 customers and the fifty with more, the split Queiroga et al. (2026) use for their own results table. Every gap in Table 4.3 uses the current BKS as the denominator, including for the published solvers. Queiroga et al. (2026) report their own average gaps against the BKS at the time of publication, effectively AILS-II’s own result for most instances, so those published averages are not comparable to the ones reported here. Appendix 2 lists the per-instance detail behind DRISPI’s two columns: instance characteristics, best cost, and gap, alongside both BKS references (Table A.1).

Table 4.3 sets DRISPI’s runs against the published solvers under different hardware and budgets. DRISPI runs 7,200 seconds of wall-clock time per instance and seed on eight pinned cores (Section 4.1.2); Queiroga et al. (2026) ran every published solver on a single thread of two AMD EPYC 9654 processors, 60 seeds of two hours per instance. Section 5.2 discusses what this difference means for the two columns of the table.

Table 4.3: DRISPI Mean Gap to Current BKS Comparison Against Benchmark Solvers, Sorted by Ascending Mean-of- N Gap Over All Instances.

Solver	Mean-of- N Gap (%)			Best-of- N Gap (%)		
	All	$n < 3,400$	$n > 3,400$	All	$n < 3,400$	$n > 3,400$
AILS-II	+0.16	+0.14	+0.18	+0.09	+0.06	+0.12
FILO2	+0.29	+0.29	+0.30	+0.20	+0.17	+0.23
DRISPI	+0.31	+0.24	+0.37	+0.28	+0.21	+0.34
FILO	+0.33	+0.31	+0.36	+0.24	+0.19	+0.28
SISRs	+0.58	+0.49	+0.68	+0.42	+0.30	+0.55
KGLSXXL	+1.09	+0.87	+1.31	+0.88	+0.64	+1.12
HGS-CVRP	+1.45	+0.83	+2.06	+1.20	+0.60	+1.80

Solver	Mean-of- N Gap (%)			Best-of- N Gap (%)		
	All	$n < 3,400$	$n > 3,400$	All	$n < 3,400$	$n > 3,400$
LKH-3 ($n = 90$)	+6.75	+5.01	+8.92	+5.56	+4.03	+7.48
OR-Tools	+6.90	+6.77	+7.03	+5.63	+5.43	+5.83

Mean-of- N is the primary comparison in Table 4.3, because DRISPI’s mean-of-3 and each published solver’s mean-of-60 both estimate the same population quantity, the expected cost of a single run, and are just averages over different sample sizes. Best-of- N is not: it is the minimum of three draws for DRISPI against the minimum of sixty for every published solver, a sampling budget 20 times larger, which mechanically favors the columns with more draws. Sampling one of DRISPI’s own three seed costs at a time gives an expected best-of-1 gap equal to the mean-of-3 gap, 0.309%; the average of the smaller cost in each of the three retained seed pairs gives a best-of-2 gap of 0.286%; the actual three-seed minimum gives the best-of-3 gap of 0.277%, already reported in Table 4.3. The marginal gain is 0.022 pp from the first extra sample and 0.009 pp from the second. AILS-II, FILO2, FILO, SISRs, KGLSXXL, and HGS-CVRP gain 0.07, 0.09, 0.10, 0.16, 0.21, and 0.25 pp respectively between their own mean-of-60 and best-of-60 columns, larger reductions from many more repetitions. DRISPI’s mean per-instance seed standard deviation is 0.035 pp, so additional seeds narrow its best column by less than they narrow a higher-variance baseline’s. This produces the rank inversion in Table 4.3: DRISPI ranks third by mean-of- N and fourth by best-of- N , without a change in the underlying solution quality; the difference is which column rewards low run-to-run variance and which rewards a wide sampling budget. Best-of- N is reported to match the published solvers’ own headline metric, not as the primary comparison.

The means in Table 4.3 do not establish whether an observed difference is real rather than an artifact of the 100 sampled instances. We tested DRISPI’s mean-of-3 column against the corresponding mean-of-60 column of five of the eight published solvers, AILS-II, FILO2, FILO, SISRs, and HGS-CVRP, with the paired Wilcoxon signed-rank test (Demšar, 2006), treating the 100 instances as matched pairs, and controlled the family-wise error rate across these five tests with the Holm step-down correction (Demšar, 2006). KGLSXXL, LKH-3, and OR-Tools are omitted: their gaps in Table 4.3 are large enough that the paired comparison would not be informative. Best-of- N compares DRISPI’s three draws against sixty for every published solver, a sampling budget twenty times larger, and is not the equal-sample-size comparison the paired test requires; it is reported below descriptively, alongside each mean-of- N result, rather than tested.

DRISPI’s mean-of-3 is significantly worse than AILS-II’s mean-of-60 (+0.150 pp, DRISPI ahead on seven of 100 instances, Holm-corrected **[TODO: NUMBER:**

Holm-corrected p , family of 5]; best-of- N , descriptive only: +0.188 pp, DRISPI ahead on one of 100) and significantly better than SISRs (-0.274 pp, Holm-corrected **[TODO: NUMBER: Holm-corrected p , family of 5]**; best-of- N , descriptive only: -0.148 pp). It is also significantly better than HGS-CVRP, by a wider margin (-1.139 pp, DRISPI ahead on all 100 instances, Holm-corrected **[TODO: NUMBER: Holm-corrected p , family of 5]**; best-of- N , descriptive only: -0.924 pp, DRISPI ahead on 96 of 100). These three verdicts do not depend on the exact corrected value: each was already significant at $p < 10^{-9}$ under the wider, ten-test family this paragraph previously reported, and the Holm correction on a five-test family, a strict subset of that ten, cannot raise a retained test's corrected p above what it already was under the larger family. Against FILO, the mean-of-3 versus mean-of-60 comparison is -0.025 pp, Holm-corrected **[TODO: NUMBER: Holm-corrected p , family of 5]** (best-of- N , descriptive only: $+0.040$ pp); against FILO2, the primary comparison, it is $+0.015$ pp, Holm-corrected **[TODO: NUMBER: Holm-corrected p , family of 5]** (best-of- N , descriptive only: $+0.074$ pp). Both were non-significant under the ten-test family; whether either crosses into significance under the smaller five-test family depends on the recomputed value and is not asserted here.

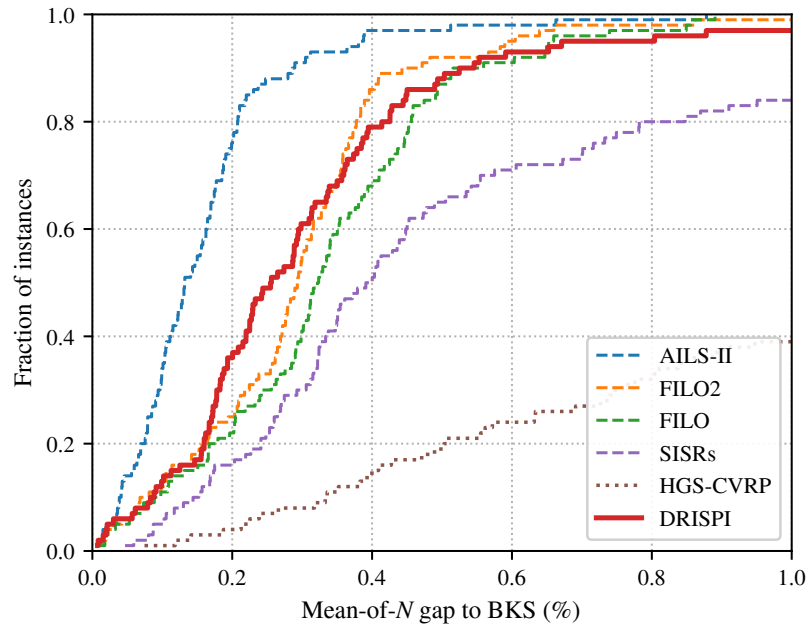


Figure 4.1: Empirical CDF of the Mean-of- N Gap to the Current BKS.

Figure 4.1 shows why the Table 4.3 means understate DRISPI's typical result and its tail at once: DRISPI's mean-of-3 gap has a mean of 0.309% against a median of 0.255% (interquartile range 0.172%–0.380%, ninetieth percentile 0.527%, maximum 1.683%), a right-skewed distribution in which most instances sit below the mean and a short tail sits well above it. DRISPI's curve tracks above FILO2's up to a gap of about 0.3% (61% of instances at or below that gap, against 55% for FILO2) and falls below it from 0.4% (79% against 86%), the crossover the non-significant

paired mean-of- N test against FILO2 allows: DRISPI wins on the tighter half of the distribution and loses on the wider half. Against FILO, DRISPI's curve stays at or above FILO's over the range shown, consistent with the near-even 53-of-100 split in the paired test. HGS-CVRP's curve sits below all five other curves across the entire range shown: only 39% of its instances reach a gap of one percent or less, and it does not visibly approach the other curves anywhere in the figure, consistent with the wide, significant margin the paired test above reports against it.

Table 4.4 lists the instances behind that tail, the largest mean-of-3 gaps in Table A.1, in descending order down to about +0.6 percent.

Table 4.4: DRISPI's Largest Mean-of-3 Gap to Current BKS (Abbr. as per Section 4.1.1).

Instance	n	Depot	Customers	Demand	r	Mean-of-3 Gap (%)
XL-n9571-k55	9,571	R	R	Q	174.7	+1.68
XL-n6034-k61	6,034	R	C (5)	5-10	98.9	+1.18
XL-n2634-k17	2,634	R	C (4)	1-10	162.6	+1.17
XL-n5174-k55	5,174	C	C (6)	1-10	94.1	+0.88
XL-n8207-k108	8,207	C	C (2)	1-10	75.9	+0.80
XL-n5902-k122	5,902	E	RC (3)	Q	48.8	+0.67
XL-n8028-k294	8,028	C	R	Q	27.3	+0.65

Five of these seven instances fall in the dataset's ultra-long route band, $r \geq 50$ (Section 4.1.1); Section 4.2.2 takes up this concentration directly.

Two individual results do not appear in any aggregate. On XL-n4535-k1134, DRISPI's best-of-3 cost of 1,203,553 is below the initial BKS of 1,203,566 Queiroga et al. (2026) reported at publication. On XL-n1094-k157, DRISPI's best-of-3 cost matches both the initial and the current BKS exactly, at 112,431. Across all 100 instances, DRISPI's best-of-3 cost matches or beats AILS-II's mean-of-60 on 12 instances, listed in Table 4.5.

Table 4.5: DRISPI’s Best-of-3 Cost Matching or Beating a Published Reference.

Instance	r	DRISPI Best-of-3	Reference Cost	Reference
XL-n1094-k157	7.8	112,431	112,431.0	Initial & current BKS
XL-n1281-k29	45.5	31,161	31,201.3	AILS-II mean-of-60
XL-n1328-k19	72.6	38,261	38,317.4	AILS-II mean-of-60
XL-n1374-k278	4.9	233,247	233,248.0	AILS-II mean-of-60
XL-n1654-k11	155.4	36,420	36,429.1	AILS-II mean-of-60
XL-n1794-k163	11.2	141,739	141,777.6	AILS-II mean-of-60
XL-n1981-k13	153.5	32,597	32,639.1	AILS-II mean-of-60
XL-n2307-k34	69.6	47,980	48,020.2	AILS-II mean-of-60
XL-n2727-k546	5.3	431,165	431,173.0	AILS-II mean-of-60
XL-n3804-k29	134.0	52,930	52,935.8	AILS-II mean-of-60
XL-n3888-k1010	4.2	1,881,512	1,882,369.0	AILS-II mean-of-60
XL-n4535-k1134	4.3	1,203,553	1,203,566.0	Initial BKS

4.2.2 Performance by Instance Characteristics

The XL instance set varies two candidate notions of scale at once: the customer count n and the target average route length r (Section 4.1.1). The two are not interchangeable stand-ins for the same underlying quantity. r is not derived from n ; it is an independent design factor of the benchmark’s own generation scheme, which fixes vehicle capacity Q and fleet size jointly so that a minimum-route solution attains the target r for whatever n the instance happens to have, so the 100-instance design deliberately crosses a wide range of r against a wide range of n rather than letting one determine the other. For DRISPI specifically, the two also act on the pipeline differently: n sets how many customers decomposition has to distribute across k subclusters, while r sets how many stops a single vehicle round trip covers on average and, by construction, compresses the number of routes relative to customers once it grows large. A regression that treats n as the only scale variable therefore risks crediting to instance size a pattern that a small number of long-route instances are actually driving.

A linear regression of the mean-of-3 gap on n over all 100 instances gives a slope of [TODO: NUMBER: gap-on- n slope, all 100 instances] (test statistic [TODO: NUMBER: gap-on- n test statistic, all 100 instances], p = [TODO: NUMBER: gap-on- n p-value, all 100 instances], 95% confidence interval [TODO: NUMBER: gap-on- n 95% CI, all 100 instances]). We define the long-route instances as those in the dataset’s own ultra-long route band, $r \geq 50$ (Section 4.1.1), a threshold fixed by the benchmark’s band definition rather than chosen after inspecting the gaps, and it separates 14 long-route instances from the remaining 86. Restricting the same regression to those 86 instances changes the slope to [TODO: NUM-

BER: gap-on- n slope, $r < 50$ only] (test statistic [TODO: NUMBER: gap-on- n test statistic, $r < 50$ only], $p =$ [TODO: NUMBER: gap-on- n p-value, $r < 50$ only], 95% confidence interval [TODO: NUMBER: gap-on- n 95% CI, $r < 50$ only]). [TODO: NUMBER: state here, once both p-values above are filled in, whether the n -slope loses significance on the $r < 50$ subset]. If it does, the scale-gap relationship measured over all 100 instances is carried substantially by the 14 long-route instances rather than by instance size on its own.

This pattern is consistent with what decomposition does to a long route rather than with a direct effect of n . Every clustering method in Table 3.2 draws its partition from customer coordinates or route centroids and the dissimilarity metric of Equation 3.4, without reference to which vehicle will eventually need which customers; a long route spans a customer set spread further apart on average than a short route serving a comparable number of stops, and that same spatial spread makes it more likely that some clustering cut line passes through the route rather than around it. Once a long route straddles a cluster boundary this way, the routing stage (Section 3.3) still solves each side of the cut to a good local optimum in isolation, but the seam between them is exactly the kind of defect BG-AILS exists to repair (Section 3.4). BG-AILS repairs every seam in an instance under the same fixed chain length of k cross-reconnects and the same wall-clock budget of Equation 3.8 regardless of how many of those seams cut through a long route rather than a short one, so an instance whose long routes cross more boundaries has more repair work competing for a budget that does not grow to match it.

Figure 4.2 sets this account against the remaining instance characteristics descriptively: depot position, customer distribution, demand distribution, and n , each summarized by the per-category mean-of-3 gap (panels a-c) or plotted directly against the per-instance gap (panel d), rather than argued from.

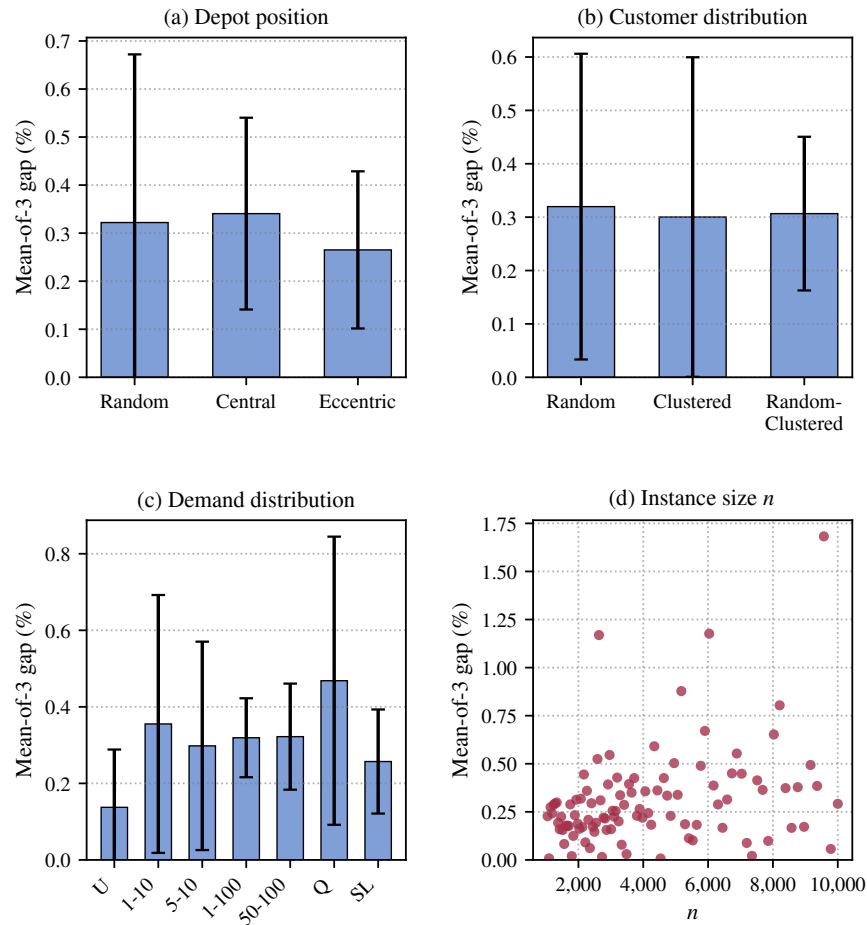


Figure 4.2: Mean-of-3 Gap to the Current BKS by Instance Characteristic, Across All 100 XL instances (Error Bars Are One Sample Standard Deviation).

4.2.3 HAOS Weights

Investigate how HAOS weights are converging and if there are obvious tendencies. Interpret the results.

Are some HAOS levels converging always to the same result? Are some HAOS levels always converging in a similar way subject to the instance characteristics? What's the variance in HAOS weight convergence? Do they differ a lot from seed to seed and are sensitive to randomness?

Are the results interpretable in general?

4.2.4 Convergence Behavior

Show how the convergence behavior is like for DRISPI by wallclock time and iterations. Use this chapter to also investigate improvement sources (quantitatively and qualitatively).

5 Conclusion

Short introduction. It should relect on the primary and secondary research questions.

5.1 Highlights

Mention the strengths (such as adaptiveness through online learning) and weaknesses (such as more CPU time required) of the framework.

Also elaborate on the main contributions of the thesis.

5.2 Limitations

Table 4.3 compares runs that differ in hardware, parallelism, and total budget, and none of its gaps correct for these differences. Queiroga et al. (2026) ran every published solver on a single thread of a machine with two AMD EPYC 9654 processors (Zen 4, 2.40 GHz), 60 seeds of two hours per instance. DRISPI ran three seeds of two hours per instance on eight pinned cores of two Intel Xeon Platinum 8280L processors (Cascade Lake, 2.70 GHz; Section 4.1.2). Both machines were shared: the published runs executed as up to 50 parallel jobs on one machine, DRISPI as six concurrent containers, three per socket, pinned to physical cores.

Accorsi et al. (2022b) recommend bringing running times from different processors to a common scale with the PassMark single-thread rating before comparing them. The EPYC 9654 rates 2,898⁷ and the Platinum 8280L 2,316⁸, a factor of 1.25. That factor is a lower bound: the 8280L rating rests on few samples and is measured at the 4.0 GHz single-core turbo, which the chip cannot hold with 24 of its 28 cores busy. Scaled this way, DRISPI's 7,200 seconds per run correspond to at most 5,800 seconds on the published solvers' processor, per core. Table 4.3 reports unscaled gaps: DRISPI was not rerun with a longer limit to offset the slower hardware, and a single-thread rating says nothing about how well eight cores are used in parallel.

The budget asymmetry points in opposite directions for the two columns of Table 4.3. Per run, DRISPI used 16 core-hours (at most 12.8 after PassMark scaling)

⁷<https://www.cpubenchmark.net/cpu.php?cpu=AMD+EPYC+9654&id=5088>, accessed September 19, 2026.

⁸<https://www.cpubenchmark.net/cpu.php?cpu=Intel+Xeon+Platinum+8280L+@+2.70GHz&id=6729>, accessed September 19, 2026.

against two CPU-hours for each published run, so the mean-of- N column compares runs in which DRISPI had eight times the cores, on older hardware. Those eight cores do not deliver eight times the search. Subcluster routing runs in waves that leave `dri` cores idle whenever the last wave holds fewer than six subclusters, the main loop blocks while it waits for BG-AILS (Figure 3.4), and three containers share the memory bandwidth and last-level cache of each socket. Per instance, the relation reverses. A published best-of-60 draws on 120 CPU-hours and 120 hours of wall-clock time, DRISPI's best-of-3 on 48 core-hours (at most 38 after PassMark scaling) and six hours. The minimum of 60 runs from a distribution lies systematically below the minimum of three, so the best-of- N column favors the published solvers by construction. Neither column is an equal-resource ranking.

The 7,200-second limit matches the published per-run budget, not the run length DRISPI needs. HAOS learns online. For the first ten iterations it draws uniformly and updates nothing (Section 3.1), and after that every draw of a weak arm still costs a full decomposition, routing, and BG-AILS pass. Under a fixed wall-clock limit, the number of completed iterations therefore bounds how far the wheels can move. That bound is tightest where one routing wave is expensive: large n and, separately, long routes. Section 4.2.4 reports the realized iteration counts and [TODO: share of runs still improving in the final 20 minutes]. Whether a longer run would close the remaining gap on these instances was not tested.

$S = 3$ was sized for the grand mean $\bar{G}$ (Section 4.1.4) and supports claims at that level only. The forecast 95% half-width of $\bar{G}$ is 0.013 pp. For a single instance, the mean of three seeds at the pilot standard deviation of 0.118 pp has a standard error of 0.068 pp and, with two degrees of freedom ($t_{0.975,2} = 4.30$), a 95% half-width of about 0.29 pp. That is more than twenty times wider, and wider still on the high-variance instances. The per-instance results in Section 4.2 (the largest gaps in Table 4.4, the instances where DRISPI's best-of-3 matches or beats a published result in Table 4.5, and [TODO: small subgroups in Section 4.2.2]) are therefore descriptive. Across 100 instances, some differences of that size will arise from the seed alone.

The seed term is also not the largest source of error. Two pilot campaigns that differed only in execution environment showed a batch effect of about 0.039 pp, three times the $S = 3$ half-width, and more seeds would not have reduced it. Shared-server load and Docker pinning are part of the measurement rather than a nuisance the design controlled.

The contribution of HAOS is established descriptively, through the weight trajectories of Section 4.2.3, and not against a control arm that freezes the draw at one configuration for the whole run under the same budget. That arm was not run. The 300-run campaign behind Section 4.2.1 used the server allocation available for this thesis, and a controlled comparison needs a full campaign run alongside it; a subset carved out of the existing allocation would confound the two. Altendeitering

and Bachmann (2026) leave the same gap for DRSCI, a DRISPI predecessor that draws paradigm, k , and solver uniformly at random each iteration without learning. Whether HAOS pays a sampling cost relative to a single committed configuration, or improves on it, remains open.

5.3 Future Research

Use the HAOS weights to train an offline learning model and inject the results back into the framework. This effectively eliminates the warm-up period and will lead to a substantial speed-up of the framework.

Also mention that the framework, other than monolithic solvers, is highly adaptable to different instances and VRP variants. Since it only requires the plug-and-play of capable clustering methods and solvers on their respective stages.

Bibliography

- Accorsi, L., F. Cavaliere, D. Vigo. 2022a. Iterative fast optimization for the Capacitated Vehicle Routing Problem. 12th DIMACS Implementation Challenge on Vehicle Routing Problems. URL <http://dimacs.rutgers.edu/programs/challenge/vrp/papers-videos/>. Team Route2Bo. Finalist paper.
- Accorsi, L., A. Lodi, D. Vigo. 2022b. Guidelines for the computational testing of machine learning approaches to vehicle routing problems. *Operations Research Letters*, 50 (2), 229-234, doi: 10.1016/j.orl.2022.01.018. URL <https://arxiv.org/abs/2109.13983>.
- Accorsi, L., D. Vigo. 2021. A fast and scalable heuristic for the solution of large-scale capacitated vehicle routing problems. *Transportation Science*, 55 (4), 832-856, doi: 10.1287/trsc.2021.1059.
- Accorsi, L., D. Vigo. 2024. Routing one million customers in a handful of minutes. *Computers & Operations Research*, 164 106562, doi: 10.1016/j.cor.2024.106562.
- Altendeitering, L., R. Bachmann. 2026. DRSCI: An algorithmic framework for very large-scale capacitated vehicle routing problems. Unpublished project study, Technical University of Munich. Chair of Logistics and Supply Chain Management; supervisor N. Kuttruff.
- Archetti, C., L. C. Coelho, M. G. Speranza, P. Vansteenwegen. 2026. Beyond fifty years of vehicle routing: Insights into the history and the future. *European Journal of Operational Research*, 330 (2), 355-372, doi: 10.1016/j.ejor.2025.06.014.
- Arenas-Vasco, A., D. Alcázar, J. G. Villegas. 2025. A meta-analysis of set partitioning/set covering based matheuristics for vehicle routing problems. *Operations Research Perspectives*, 15 100357, doi: 10.1016/j.orp.2025.100357.
- Arnold, F., M. Gendreau, K. Sörensen. 2019. Efficiently solving very large-scale routing problems. *Computers & Operations Research*, 107 32-42, doi: 10.1016/j.cor.2019.03.006.
- Arnold, F., K. Sörensen. 2019. Knowledge-guided local search for the vehicle routing problem. *Computers & Operations Research*, 105 32-46, doi: 10.1016/j.cor.2019.01.002.
- Auer, P., N. Cesa-Bianchi, P. Fischer. 2002. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47 (2-3), 235-256, doi: 10.1023/a:1013689704352.

- Balinski, M. L., R. E. Quandt. 1964. On an integer program for a delivery problem. *Operations Research*, 12 (2), 300-304, doi: 10.1287/opre.12.2.300.
- Barr, R. S., B. L. Golden, J. P. Kelly, M. G. C. Resende, W. R. Stewart. 1995. Designing and reporting on computational experiments with heuristic methods. *Journal of Heuristics*, 1 (1), 9-32, doi: 10.1007/bf02430363.
- Beasley, J. E. 1983. Route first—cluster second methods for vehicle routing. *Omega*, 11 (4), 403-408, doi: 10.1016/0305-0483(83)90033-6.
- Bengio, Y., A. Lodi, A. Prouvost. 2021. Machine learning for combinatorial optimization: A methodological tour d’horizon. *European Journal of Operational Research*, 290 (2), 405-421, doi: 10.1016/j.ejor.2020.07.063.
- Bent, R., P. Van Hentenryck. 2010. Spatial, temporal, and hybrid decompositions for large-scale vehicle routing with time windows. David Cohen, ed., *Principles and Practice of Constraint Programming — CP 2010, Lecture Notes in Computer Science*, vol. 6308. Springer, Berlin, Heidelberg, 99-113, doi: 10.1007/978-3-642-15396-9_11.
- Bezdek, J. C., R. Ehrlich, W. Full. 1984. FCM: The fuzzy c-means clustering algorithm. *Computers & Geosciences*, 10 (2-3), 191-203, doi: 10.1016/0098-3004(84)90020-7.
- Burke, E. K., M. Gendreau, M. Hyde, G. Kendall, G. Ochoa, E. Özcan, R. Qu. 2013. Hyper-heuristics: a survey of the state of the art. *Journal of the Operational Research Society*, 64 (12), 1695-1724, doi: 10.1057/jors.2013.71.
- Cavaliere, F., E. Bendotti, M. Fischetti. 2022. An integrated local-search/set-partitioning refinement heuristic for the Capacitated Vehicle Routing Problem. *Mathematical Programming Computation*, 14 (4), 749-779, doi: 10.1007/s12532-022-00224-2.
- Christiaens, J., G. Vanden Berghe. 2020. Slack Induction by String Removals for Vehicle Routing Problems. *Transportation Science*, 54 (2), 417-433, doi: 10.1287/trsc.2019.0914.
- Cochran, W. G. 1977. *Sampling Techniques*. 3rd ed. Wiley, New York.
- Costa, L., C. Contardo, G. Desaulniers. 2019. Exact branch-price-and-cut algorithms for vehicle routing. *Transportation Science*, 53 (4), 946-985, doi: 10.1287/trsc.2018.0878.
- Crainic, T. G. 2008. Parallel solution methods for vehicle routing problems. Bruce Golden, S. Raghavan, Edward Wasil, eds., *The Vehicle Routing Problem: Latest Advances and New Challenges*. Springer, Boston, MA, 171-198, doi: 10.1007/978-0-387-77778-8_8.
- Crainic, T. G. 2019. Parallel metaheuristics and cooperative search. Michel Gendreau, Jean-Yves Potvin, eds., *Handbook of Metaheuristics, International Se-*

- ries in *Operations Research & Management Science*, vol. 272, 3rd ed. Springer, Cham, 419-451, doi: 10.1007/978-3-319-91086-4_13.
- Dantzig, G. B., J. H. Ramser. 1959. The truck dispatching problem. *Management Science*, 6 (1), 80-91, doi: 10.1287/mnsc.6.1.80.
- Demšar, J. 2006. Statistical Comparisons of Classifiers over Multiple Data Sets. *Journal of Machine Learning Research*, 7 (1), 1-30. URL <https://www.jmlr.org/papers/v7/demsar06a.html>.
- Eiben, A. E., R. Hinterding, Z. Michalewicz. 1999. Parameter control in evolutionary algorithms. *IEEE Transactions on Evolutionary Computation*, 3 (2), 124-141, doi: 10.1109/4235.771166.
- Fialho, Á., L. Da Costa, M. Schoenauer, M. Sebag. 2010. Analyzing bandit-based adaptive operator selection mechanisms. *Annals of Mathematics and Artificial Intelligence*, 60 (1-2), 25-64, doi: 10.1007/s10472-010-9213-y.
- Fisher, M. L., R. Jaikumar. 1981. A generalized assignment heuristic for vehicle routing. *Networks*, 11 (2), 109-124, doi: 10.1002/net.3230110205.
- Foster, B. A., D. M. Ryan. 1976. An integer programming approach to the vehicle scheduling problem. *Journal of the Operational Research Society*, 27 (2), 367-384, doi: 10.1057/jors.1976.63. Published under the journal's former title, *Operational Research Quarterly*.
- Garfinkel, R. S., G. L. Nemhauser. 1969. The set-partitioning problem: Set covering with equality constraints. *Operations Research*, 17 (5), 848-856, doi: 10.1287/opre.17.5.848.
- Gillett, B. E., L. R. Miller. 1974. A heuristic algorithm for the vehicle-dispatch problem. *Operations Research*, 22 (2), 340-349, doi: 10.1287/opre.22.2.340.
- Groër, C., B. Golden, E. Wasil. 2011. A Parallel Algorithm for the Vehicle Routing Problem. *INFORMS Journal on Computing*, 23 (2), 315-330, doi: 10.1287/ijoc.1100.0402.
- Helsgaun, K. 2017. An extension of the lin-kernighan-helsgaun TSP solver for constrained traveling salesman and vehicle routing problems. Technical report, Roskilde University, Roskilde. URL http://webhotel4.ruc.dk/~keld/research/LKH-3/LKH-3_REPORT.pdf.
- Hou, Q., J. Yang, Y. Su, X. Wang, Y. Deng. 2023. Generalize learned heuristics to solve large-scale vehicle routing problems in real-time. *International Conference on Learning Representations*. URL <https://openreview.net/forum?id=6ZajpxqTlQ>.
- Jaccard, P. 1901. Distribution de la flore alpine dans le bassin des dranses et dans quelques régions voisines. *Bulletin de la Société Vaudoise des Sciences Naturelles*, 37 241-272, doi: 10.5169/seals-266440.

- Jin, J., T. G. Crainic, A. Løkketangen. 2014. A cooperative parallel metaheuristic for the capacitated vehicle routing problem. *Computers & Operations Research*, 44 33-41, doi: 10.1016/j.cor.2013.10.004.
- Johnson, D. S. 2002. A theoretician's guide to the experimental analysis of algorithms. Michael H. Goldwasser, David S. Johnson, Catherine C. McGeoch, eds., *Data Structures, Near Neighbor Searches, and Methodology: Fifth and Sixth DIMACS Implementation Challenges, DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, vol. 59. American Mathematical Society, Providence, RI, 215-250, doi: 10.1090/dimacs/059/11.
- Jourdan, L., M. Basseur, E.-G. Talbi. 2009. Hybridizing exact methods and metaheuristics: A taxonomy. *European Journal of Operational Research*, 199 (3), 620-629, doi: 10.1016/j.ejor.2007.07.035.
- Kallestad, J., R. Hasibi, A. Hemmati, K. Sørensen. 2023. A general deep reinforcement learning hyperheuristic framework for solving combinatorial optimization problems. *European Journal of Operational Research*, 309 (1), 446-468, doi: 10.1016/j.ejor.2023.01.017.
- Karaahmetoglu, O., H. Kim. 2026. Adaptive cluster-first route-second decomposition for industrial-scale vehicle routing. doi: 10.48550/arxiv.2606.31820. ArXiv:2606.31820.
- Karimi-Mamaghan, M., M. Mohammadi, P. Meyer, A. M. Karimi-Mamaghan, E.-G. Talbi. 2022. Machine learning at the service of meta-heuristics for solving combinatorial optimization problems: A state-of-the-art. *European Journal of Operational Research*, 296 (2), 393-422, doi: 10.1016/j.ejor.2021.04.032.
- Kaufman, L., P. J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley Series in Probability and Statistics, John Wiley & Sons, New York, doi: 10.1002/9780470316801.
- Kelly, J. P., J. Xu. 1999. A set-partitioning-based heuristic for the vehicle routing problem. *INFORMS Journal on Computing*, 11 (2), 161-172, doi: 10.1287/ijoc.11.2.161.
- Kerschler, C., S. Minner. 2025. Decompose-route-improve framework for solving large-scale vehicle routing problems with time windows. *Transportation Research Part E: Logistics and Transportation Review*, 204 104409, doi: 10.1016/j.tre.2025.104409.
- Kerschler, C., S. Minner, F. Lehuédé, G. Massonnet. 2026. Solving heterogeneous vehicle routing problems using decomposition and set covering. *Computers & Operations Research*, 193 107505, doi: 10.1016/j.cor.2026.107505.
- Kerschke, P., H. H. Hoos, F. Neumann, H. Trautmann. 2019. Automated algorithm selection: Survey and perspectives. *Evolutionary Computation*, 27 (1), 3-45, doi: 10.1162/evco_a_00242.

- Kool, W., H. van Hoof, M. Welling. 2019. Attention, learn to solve routing problems! *7th International Conference on Learning Representations (ICLR 2019)*. New Orleans, LA. URL <https://openreview.net/forum?id=ByxBFsRqYm>.
- Kytöjoki, J., T. Nuortio, O. Bräysy, M. Gendreau. 2007. An efficient variable neighborhood search heuristic for very large scale vehicle routing problems. *Computers & Operations Research*, 34 (9), 2743-2757, doi: 10.1016/j.cor.2005.10.010.
- Law, A. M., W. D. Kelton. 2000. *Simulation Modeling and Analysis*. 3rd ed. McGraw-Hill, Boston.
- Lenstra, J. K., A. H. G. Rinnooy Kan. 1981. Complexity of vehicle routing and scheduling problems. *Networks*, 11 (2), 221-227, doi: 10.1002/net.3230110211.
- Li, S., Z. Yan, C. Wu. 2021. Learning to delegate for large-scale vehicle routing. *Advances in Neural Information Processing Systems*, vol. 34. 26198-26211. URL <https://proceedings.neurips.cc/paper/2021/hash/dc9fa5f217a1e57b8a6adeb065560b38-Abstract.html>.
- López-Ibáñez, M., J. Dubois-Lacoste, L. Pérez Cáceres, M. Birattari, T. Stützle. 2016. The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3 43-58, doi: 10.1016/j.orp.2016.09.002.
- Lourenço, H. R., O. C. Martin, T. Stützle. 2003. Iterated Local Search. Fred Glover, Gary A. Kochenberger, eds., *Handbook of Metaheuristics*. Springer US, Boston, MA, 320-353, doi: 10.1007/0-306-48056-5_11.
- Lübbecke, M. E., J. Desrosiers. 2005. Selected Topics in Column Generation. *Operations Research*, 53 (6), 1007-1023, doi: 10.1287/opre.1050.0234.
- MacQueen, J. B. 1967. Some methods for classification and analysis of multivariate observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1. University of California Press, Berkeley, CA, 281-297. URL <https://projecteuclid.org/euclid.bsmssp/1200512992>.
- Martin, O., S. W. Otto, E. W. Felten. 1991. Large-step Markov chains for the Traveling Salesman Problem. *Complex Systems*, 5 (3), 299-326.
- Máximo, V. R., J.-F. Cordeau, M. C. V. Nascimento. 2024. AILS-II: An adaptive iterated local search heuristic for the large-scale capacitated vehicle routing problem. *INFORMS Journal on Computing*, 36 (4), 974-986, doi: 10.1287/i-joc.2023.0106.
- Máximo, V. R., M. C. V. Nascimento. 2021. A hybrid adaptive iterated local search with diversification control to the capacitated vehicle routing problem. *European Journal of Operational Research*, 294 (3), 1108-1119, doi: 10.1016/j.ejor.2021.02.024.

- Neyman, J. 1934. On the two different aspects of the representative method: The method of stratified sampling and the method of purposive selection. *Journal of the Royal Statistical Society*, 97 (4), 558-625, doi: 10.2307/2342192.
- Pecin, D., A. A. Pessoa, M. Poggi, E. Uchoa. 2017. Improved branch-cut-and-price for capacitated vehicle routing. *Mathematical Programming Computation*, 9 (1), 61-100, doi: 10.1007/s12532-016-0108-8.
- Pessoa, A. A., R. Sadykov, E. Uchoa, F. Vanderbeck. 2020. A generic exact solver for vehicle routing and related problems. *Mathematical Programming*, 183 483-523, doi: 10.1007/s10107-020-01523-z.
- Pisinger, D., S. Ropke. 2007. A general heuristic for vehicle routing problems. *Computers & Operations Research*, 34 (8), 2403-2435, doi: 10.1016/j.cor.2005.09.012.
- Prins, C. 2004. A simple and effective evolutionary algorithm for the vehicle routing problem. *Computers & Operations Research*, 31 (12), 1985-2002, doi: 10.1016/s0305-0548(03)00158-8.
- Queiroga, E., R. Martinelli, A. Subramanian, E. Uchoa, T. Vidal. 2026. The XL instances and the CVRPLib best known solution challenge. doi: 10.48550/arXiv.2601.11467. Version 2. Version 1 appeared as "The XL Instances for the Capacitated Vehicle Routing Problem".
- Queiroga, E., R. Sadykov, E. Uchoa. 2021. A POPMUSIC matheuristic for the capacitated vehicle routing problem. *Computers & Operations Research*, 136 105475, doi: 10.1016/j.cor.2021.105475.
- Queiroga, E., R. Sadykov, E. Uchoa, T. Vidal. 2022. 10,000 optimal CVRP solutions for testing machine learning based heuristics. *AAAI-22 Workshop on Machine Learning for Operations Research (ML4OR)*. URL <https://openreview.net/forum?id=yHiMXKN6nTl>. Preprint available as arXiv:2109.13983.
- Reijnen, R., Y. Zhang, H. C. Lau, Z. Bukhsh. 2024. Online control of adaptive large neighborhood search using deep reinforcement learning. *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 34. 475-483, doi: 10.1609/icaps.v34i1.31507.
- Rochat, Y., E. D. Taillard. 1995. Probabilistic diversification and intensification in local search for vehicle routing. *Journal of Heuristics*, 1 (1), 147-167, doi: 10.1007/bf02430370.
- Ropke, S., D. Pisinger. 2006. An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows. *Transportation Science*, 40 (4), 455-472, doi: 10.1287/trsc.1050.0135.
- Santini, A., M. Schneider, T. Vidal, D. Vigo. 2023. Decomposition Strategies for Vehicle Routing Heuristics. *INFORMS Journal on Computing*, 35 (3), 543-559, doi: 10.1287/ijoc.2023.1288.

- Sörensen, K. 2015. Metaheuristics—the metaphor exposed. *International Transactions in Operational Research*, 22 (1), 3-18, doi: 10.1111/itor.12001.
- Sörensen, K., M. Sevaux, F. Glover. 2018. A history of metaheuristics. Rafael Martí, Panos M. Pardalos, Mauricio G. C. Resende, eds., *Handbook of Heuristics*. Springer, Cham, 791-808, doi: 10.1007/978-3-319-07124-4_4.
- Subramanian, A., E. Uchoa, L. S. Ochi. 2013. A hybrid algorithm for a class of vehicle routing problems. *Computers & Operations Research*, 40 (10), 2519-2531, doi: 10.1016/j.cor.2013.01.013.
- Taillard, É. D. 1993. Parallel iterative search methods for vehicle routing problems. *Networks*, 23 (8), 661-673, doi: 10.1002/net.3230230804.
- Taillard, É. D., L. M. Gambardella, M. Gendreau, J.-Y. Potvin. 2001. Adaptive memory programming: A unified view of meta-heuristics. *European Journal of Operational Research*, 135 (1), 1-16, doi: 10.1016/S0377-2217(00)00268-X.
- Taillard, É. D., S. Voß. 2002. POPMUSIC — partial optimization metaheuristic under special intensification conditions. Celso C. Ribeiro, Pierre Hansen, eds., *Essays and Surveys in Metaheuristics, Operations Research/Computer Science Interfaces Series*, vol. 15. Kluwer Academic Publishers, Boston, MA, 613-629, doi: 10.1007/978-1-4615-1507-4_27.
- Toth, P., D. Vigo. 2003. The granular tabu search and its application to the vehicle-routing problem. *INFORMS Journal on Computing*, 15 (4), 333-346, doi: 10.1287/ijoc.15.4.333.24890.
- Toth, P., D. Vigo, eds. 2014. *Vehicle Routing: Problems, Methods, and Applications*. 2nd ed. Society for Industrial and Applied Mathematics, Philadelphia, PA, doi: 10.1137/1.9781611973594.
- Turkeš, R., K. Sörensen, L. M. Hvattum. 2021. Meta-analysis of metaheuristics: Quantifying the effect of adaptiveness in adaptive large neighborhood search. *European Journal of Operational Research*, 292 (2), 423-442, doi: 10.1016/j.ejor.2020.10.045.
- Uchoa, E., D. Pecin, A. Pessoa, M. Poggi, T. Vidal, A. Subramanian. 2017. New benchmark instances for the Capacitated Vehicle Routing Problem. *European Journal of Operational Research*, 257 (3), 845-858, doi: 10.1016/j.ejor.2016.08.012.
- Vidal, T. 2022. Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood. *Computers & Operations Research*, 140 105643, doi: 10.1016/j.cor.2021.105643.
- Vidal, T., T. G. Crainic, M. Gendreau, N. Lahrichi, W. Rei. 2012. A Hybrid Genetic Algorithm for Multidepot and Periodic Vehicle Routing Problems. *Operations Research*, 60 (3), 611-624, doi: 10.1287/opre.1120.1048.

- Vidal, T., G. Laporte, P. Matl. 2020. A concise guide to existing and emerging vehicle routing problem variants. *European Journal of Operational Research*, 286 (2), 401-416, doi: 10.1016/j.ejor.2019.10.010.
- Voigt, S. 2025. A review and ranking of operators in adaptive large neighborhood search for vehicle routing problems. *European Journal of Operational Research*, 322 (2), 357-375, doi: 10.1016/j.ejor.2024.05.033.
- Windras Mara, S. T., R. Norcahyo, P. Jodiawan, L. Lusiantoro, A. P. Rifai. 2022. A survey of adaptive large neighborhood search algorithms and applications. *Computers & Operations Research*, 146 105903, doi: 10.1016/j.cor.2022.105903.
- Wouda, N. A., L. Lan, W. Kool. 2024. PyVRP: A high-performance VRP solver package. *INFORMS Journal on Computing*, 36 (4), 943-955, doi: 10.1287/i-joc.2023.0055.
- Ye, H., J. Wang, H. Liang, Z. Cao, Y. Li, F. Li. 2024. GLOP: Learning global partition and local construction for solving large-scale routing problems in real-time. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38 (18), 20284-20292, doi: 10.1609/aaai.v38i18.30009.

Appendix

1 Set Covering Formulation

Section 3.5 introduces set covering (SC) as the fallback DRISPI runs in place of set partitioning (SP) when the route pool does not yet cover every customer at least a minimum number of times. With R the route pool at the time of the solve, c_r the cost of route $r \in R$, C the customer set of the instance (Section 2.1), $C_r \subseteq C$ the customer set of route r , and $x_r \in \{0, 1\}$ selecting route r , SC relaxes the partition equality of SP (Equation 3.12) to the cover inequality of Garfinkel and Nemhauser (1969), who characterize set partitioning as set covering under an equality constraint:

$$\min \sum_{r \in R} c_r x_r \quad (1)$$

$$\text{s.t.} \quad \sum_{r \in R: i \in C_r} x_r \geq 1, \quad \forall i \in C \quad (2)$$

$$x_r \in \{0, 1\}, \quad \forall r \in R. \quad (3)$$

As with SP, DRISPI solves the LP relaxation of Equation 3, $x_r \in [0, 1]$, first; the resulting fractional weights update the pool's quality scores in the same way regardless of whether the solve ends up running SC or SP, and the binary re-solve is warm-started from that relaxation.

2 XL Instance Detailed Results

Table A.1 reports the per-instance detail underlying the Section 4.2.1 comparison: instance characteristics, DRISPI’s best cost and gap to the current BKS for all 100 XL instances, alongside the initial-BKS and current-BKS reference costs.

Table A.1: Instance-Level Best-of-3 Results for DRISPI on the 100 XL Instances.

#	Name	Dep	Cust	Dem	Q	r	DRISPI		Initial BKS		BKS
							Cost	Gap (%)	Cost	Gap (%)	Cost
1	XL-n1048-k237	E	R	SL	128	4.7	380,838	+0.19	380,211	+0.03	380,107
2	XL-n1094-k157	C	C (6)	U	7	7.8	112,431	+0.00	112,431	+0.00	112,431
3	XL-n1141-k112	R	RC (3)	50-100	761	10.2	95,888	+0.26	95,727	+0.09	95,638
4	XL-n1188-k96	R	RC (3)	Q	782	12.4	104,600	+0.19	104,415	+0.01	104,402
5	XL-n1234-k55	E	R	1-10	126	22.7	96,848	+0.25	96,647	+0.05	96,603
6	XL-n1281-k29	C	C (5)	1-100	2267	45.5	31,161	+0.25	31,101	+0.06	31,083
7	XL-n1328-k19	R	RC (3)	5-10	542	72.6	38,261	+0.16	38,247	+0.12	38,201
8	XL-n1374-k278	C	R	Q	248	4.9	233,247	+0.13	233,049	+0.05	232,940
9	XL-n1421-k232	E	C (4)	1-100	309	6.1	385,182	+0.14	384,826	+0.05	384,646
10	XL-n1468-k151	E	R	50-100	726	9.7	250,530	+0.20	250,166	+0.06	250,028
11	XL-n1514-k106	C	RC (4)	5-10	107	14.2	92,533	+0.15	92,425	+0.03	92,398
12	XL-n1561-k75	R	C (4)	U	21	21.9	101,602	+0.06	101,549	+0.01	101,540
13	XL-n1608-k39	C	RC (3)	SL	337	42.1	48,067	+0.12	48,021	+0.02	48,009
14	XL-n1654-k11	E	R	1-10	845	155.4	36,420	+0.14	36,385	+0.04	36,370
15	XL-n1701-k562	R	C (6)	50-100	227	3.0	521,691	+0.16	521,136	+0.06	520,843
16	XL-n1748-k271	R	C (3)	Q	270	6.4	174,235	+0.27	173,896	+0.08	173,759
17	XL-n1794-k163	C	R	U	11	11.2	141,739	+0.01	141,729	+0.00	141,724
18	XL-n1841-k126	E	RC (2)	SL	186	14.6	214,184	+0.09	214,038	+0.03	213,981
19	XL-n1888-k82	E	R	5-10	173	23.1	143,864	+0.21	143,623	+0.05	143,558
20	XL-n1934-k46	C	C (6)	1-100	2166	42.2	53,080	+0.19	53,013	+0.06	52,980
21	XL-n1981-k13	R	RC (2)	1-10	832	153.5	32,597	+0.16	32,580	+0.10	32,546
22	XL-n2028-k617	C	C (6)	50-100	247	3.3	544,854	+0.13	544,403	+0.04	544,166
23	XL-n2074-k264	E	R	1-100	401	7.8	422,507	+0.30	421,627	+0.09	421,250
24	XL-n2121-k186	R	RC (5)	1-10	62	11.4	283,579	+0.17	283,211	+0.04	283,105
25	XL-n2168-k138	C	R	Q	800	15.7	127,763	+0.43	127,298	+0.07	127,210
26	XL-n2214-k131	R	C (5)	U	17	17.8	154,730	+0.05	154,676	+0.02	154,652
27	XL-n2261-k54	E	RC (5)	5-10	319	42.5	99,135	+0.31	98,907	+0.08	98,826
28	XL-n2307-k34	C	R	SL	479	69.6	47,980	+0.09	47,958	+0.05	47,936
29	XL-n2354-k631	E	C (4)	5-10	28	3.7	941,156	+0.06	940,825	+0.02	940,634
30	XL-n2401-k408	R	RC (4)	Q	303	5.9	464,406	+0.29	463,473	+0.08	463,084
31	XL-n2447-k290	C	RC (2)	SL	150	8.4	218,961	+0.15	218,706	+0.03	218,633
32	XL-n2494-k194	E	C (2)	1-100	661	12.9	361,553	+0.14	361,205	+0.04	361,057
33	XL-n2541-k121	R	R	U	21	21.7	146,609	+0.18	146,390	+0.03	146,345
34	XL-n2587-k66	C	R	50-100	2986	39.6	73,598	+0.47	73,394	+0.19	73,254
35	XL-n2634-k17	R	C (4)	1-10	898	162.6	31,946	+1.03	31,641	+0.07	31,619
36	XL-n2681-k540	E	RC (3)	1-100	251	5.0	800,308	+0.30	798,603	+0.09	797,906
37	XL-n2727-k546	C	RC (5)	U	5	5.3	431,165	+0.01	431,134	+0.01	431,108
38	XL-n2774-k286	E	C (5)	50-100	731	9.7	408,453	+0.21	407,847	+0.06	407,617
39	XL-n2821-k208	R	R	SL	179	13.5	217,020	+0.18	216,763	+0.07	216,622
40	XL-n2867-k120	R	C (4)	5-10	180	23.9	166,120	+0.12	165,990	+0.04	165,916
41	XL-n2914-k95	C	RC (3)	Q	1663	30.8	89,247	+0.39	88,990	+0.10	88,902
42	XL-n2961-k55	E	R	1-10	297	53.8	108,475	+0.49	108,084	+0.13	107,945
43	XL-n3007-k658	C	R	1-10	25	4.4	522,806	+0.13	522,319	+0.04	522,126
44	XL-n3054-k461	E	RC (4)	50-100	497	6.6	784,094	+0.24	782,739	+0.07	782,194
45	XL-n3101-k311	R	C (3)	SL	159	10.0	246,297	+0.22	245,937	+0.07	245,766
46	XL-n3147-k232	R	RC (6)	5-10	102	13.6	257,002	+0.23	256,626	+0.08	256,417
47	XL-n3194-k161	C	R	Q	1012	19.9	149,197	+0.40	148,728	+0.09	148,596
48	XL-n3241-k115	E	C (4)	1-100	1404	28.3	221,653	+0.18	221,370	+0.05	221,249
49	XL-n3287-k30	C	C (2)	U	111	112.0	40,315	+0.31	40,229	+0.10	40,190
50	XL-n3334-k934	E	R	1-10	20	3.5	1,453,320	+0.07	1,452,698	+0.02	1,452,353

Table A.1 – continued from previous page

#	Name	Dep	Cust	Dem	Q	r	DRISPI		Initial BKS		BKS
							Cost	Gap (%)	Cost	Gap (%)	Cost
51	XL-n3408-k524	R	RC (3)	Q	353	6.5	679,883	+0.27	678,643	+0.09	678,024
52	XL-n3484-k436	E	C (6)	U	8	8.2	703,486	+0.02	703,355	+0.01	703,319
53	XL-n3561-k229	C	RC (5)	1-100	779	15.6	210,003	+0.39	209,386	+0.09	209,191
54	XL-n3640-k211	R	R	5-10	130	17.2	190,170	+0.34	189,724	+0.10	189,531
55	XL-n3721-k77	E	RC (2)	SL	371	48.8	163,281	+0.40	162,862	+0.14	162,630
56	XL-n3804-k29	C	R	50-100	10064	134.0	52,930	+0.17	52,885	+0.09	52,838
57	XL-n3888-k1010	R	C (2)	SL	128	4.2	1,881,512	+0.23	1,880,368	+0.17	1,877,119
58	XL-n3975-k687	C	RC (4)	1-10	32	5.6	526,648	+0.21	525,901	+0.06	525,560
59	XL-n4063-k347	E	R	Q	598	11.7	550,156	+0.33	548,931	+0.11	548,330
60	XL-n4153-k291	R	C (3)	1-100	726	14.3	356,629	+0.24	356,034	+0.07	355,780
61	XL-n4245-k203	R	R	U	21	21.0	229,919	+0.17	229,659	+0.06	229,521
62	XL-n4340-k148	E	RC (6)	50-100	2204	29.3	245,204	+0.54	244,226	+0.14	243,895
63	XL-n4436-k48	C	C (4)	5-10	706	94.1	61,636	+0.33	61,477	+0.07	61,431
64	XL-n4535-k1134	R	C (4)	U	4	4.3	1,203,553	+0.00	1,203,566	+0.00	1,203,512
65	XL-n4635-k790	C	RC (4)	1-100	294	5.9	612,069	+0.38	610,650	+0.15	609,734
66	XL-n4738-k487	E	R	Q	499	9.7	762,041	+0.31	760,501	+0.11	759,649
67	XL-n4844-k321	R	R	SL	188	15.1	405,321	+0.22	404,652	+0.06	404,429
68	XL-n4951-k203	E	RC (5)	50-100	1848	24.5	286,170	+0.49	285,269	+0.18	284,769
69	XL-n5061-k184	C	C (5)	5-10	206	27.4	161,968	+0.30	161,629	+0.09	161,480
70	XL-n5174-k55	C	C (6)	1-10	520	94.1	61,776	+0.85	61,382	+0.21	61,254
71	XL-n5288-k1246	E	R	50-100	318	4.2	1,961,708	+0.18	1,960,101	+0.10	1,958,205
72	XL-n5406-k783	R	RC (2)	1-10	38	6.8	1,041,209	+0.10	1,040,536	+0.04	1,040,141
73	XL-n5526-k553	R	C (3)	U	10	10.0	337,155	+0.09	336,898	+0.02	336,843
74	XL-n5649-k401	E	R	SL	181	14.1	645,503	+0.15	644,866	+0.05	644,562
75	XL-n5774-k290	C	RC (4)	1-100	1012	19.9	251,102	+0.48	250,207	+0.12	249,895
76	XL-n5902-k122	E	RC (3)	Q	2663	48.8	218,441	+0.66	217,447	+0.20	217,012
77	XL-n6034-k61	R	C (5)	5-10	744	98.9	64,945	+1.08	64,448	+0.31	64,251
78	XL-n6168-k1922	C	R	1-100	162	3.2	1,533,229	+0.38	1,530,010	+0.17	1,527,390
79	XL-n6305-k1042	R	RC (2)	Q	268	6.0	1,179,220	+0.27	1,177,528	+0.13	1,176,002
80	XL-n6445-k628	E	R	5-10	77	10.2	997,672	+0.16	996,623	+0.05	996,117
81	XL-n6588-k473	C	C (4)	1-10	76	13.8	334,751	+0.28	334,068	+0.07	333,822
82	XL-n6734-k330	R	RC (2)	50-100	1534	20.4	449,273	+0.40	448,031	+0.12	447,486
83	XL-n6884-k148	E	C (4)	SL	357	46.4	182,472	+0.49	181,809	+0.12	181,585
84	XL-n7037-k38	C	R	U	187	187.1	70,981	+0.31	70,845	+0.12	70,760
85	XL-n7193-k1683	E	RC (2)	5-10	32	4.2	2,960,620	+0.08	2,958,979	+0.03	2,958,205
86	XL-n7353-k1471	R	R	U	5	5.1	1,537,979	+0.02	1,537,811	+0.01	1,537,702
87	XL-n7516-k859	C	C (2)	1-100	439	8.8	575,245	+0.39	573,902	+0.16	572,984
88	XL-n7683-k602	R	RC (3)	50-100	957	12.8	703,448	+0.33	702,098	+0.14	701,151
89	XL-n7854-k365	E	C (2)	SL	223	21.5	659,690	+0.09	659,221	+0.02	659,075
90	XL-n8028-k294	C	R	Q	1386	27.3	267,910	+0.61	266,900	+0.23	266,295
91	XL-n8207-k108	C	C (2)	1-10	415	75.9	118,763	+0.60	118,274	+0.19	118,050
92	XL-n8389-k2028	E	RC (4)	1-100	208	4.1	3,365,000	+0.35	3,358,731	+0.16	3,353,305
93	XL-n8575-k1297	R	R	1-10	36	6.5	1,090,089	+0.16	1,089,137	+0.07	1,088,376
94	XL-n8766-k1032	R	RC (4)	50-100	637	8.5	908,402	+0.37	906,406	+0.15	905,060
95	XL-n8960-k634	E	C (5)	5-10	106	14.1	774,081	+0.15	773,383	+0.06	772,906
96	XL-n9160-k379	C	R	SL	237	24.2	325,066	+0.45	324,092	+0.15	323,596
97	XL-n9363-k209	C	RC (2)	U	45	45.2	206,063	+0.34	205,575	+0.10	205,369
98	XL-n9571-k55	R	R	Q	8773	174.7	108,022	+1.58	106,791	+0.42	106,344
99	XL-n9784-k2774	E	C (4)	1-10	19	3.4	4,078,780	+0.05	4,078,217	+0.03	4,076,936
100	XL-n10001-k1570	E	RC (2)	50-100	479	6.4	2,336,473	+0.25	2,333,757	+0.13	2,330,666

Declaration Of Authorship

I hereby solemnly declare that I have completed the present work independently. All ideas taken directly or indirectly from external sources have been clearly identified as such.

I am aware that the work may be examined in digital form to determine whether unauthorized aids were used and whether it constitutes - either in whole or in part - plagiarism. For the purpose of comparison with existing sources, my work may be entered into a database and, after review, may remain there for comparison with future submissions. No further reproduction or usage rights are thereby granted. This work has neither been submitted to another examination authority nor published.

Place, Date

Lucca Altendeitering

Declaration of AI Use

Download the Declaration of AI Use from the website, complete it truthfully, and include it as a separate page at the end of your final thesis PDF here instead of this page.